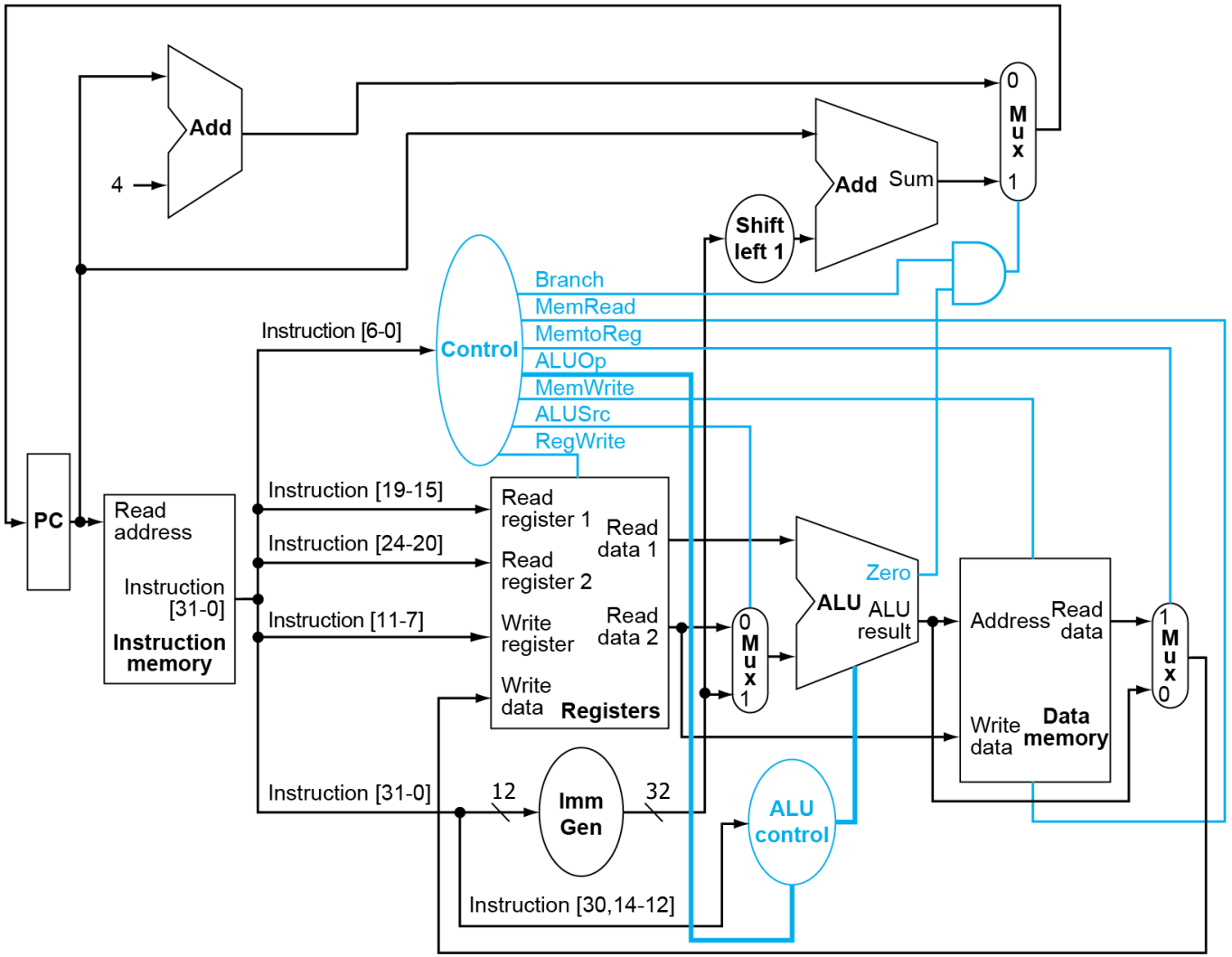
Pipelining

Instructor: Jeongseob Ahn (안정섭)

E-mail: jsahn@csl.korea.ac.kr
Hompage: https://jeongseob.github.io

[Last Lecture] Datapath with Control



[Last Lecture] Performance Issues

- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast
- We will improve performance by pipelining

Outline

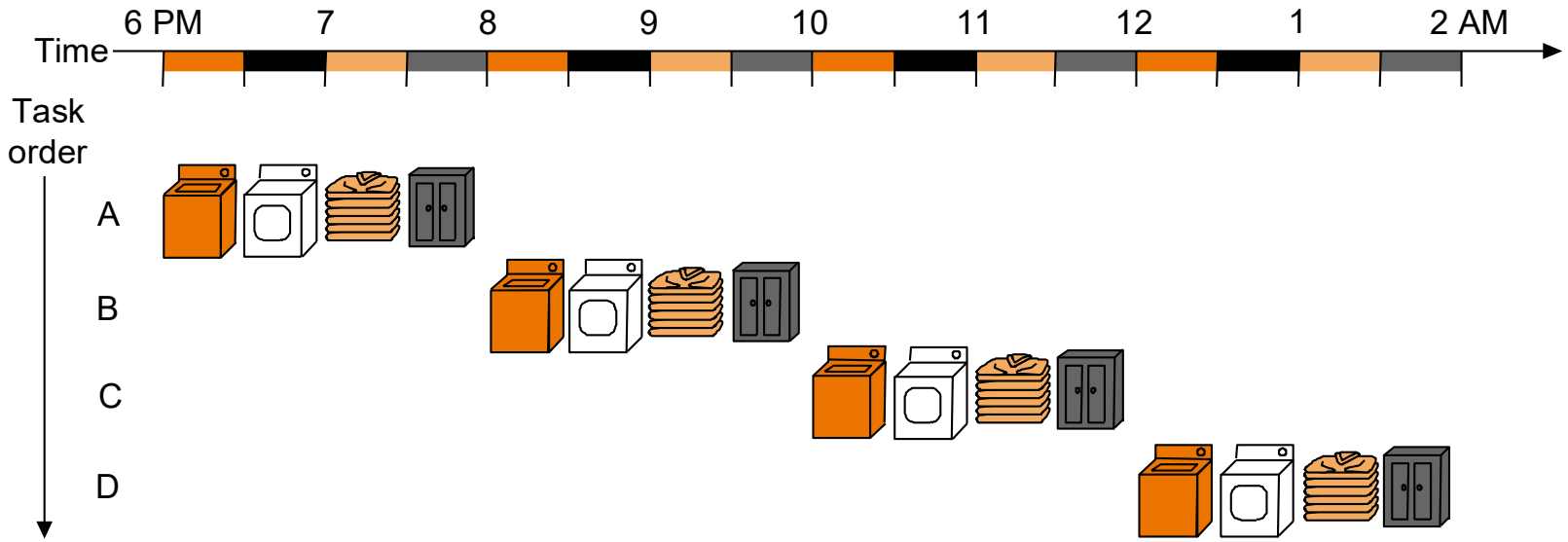
- Basic Pipelining
- Pipeline Hazards
  - Structure, Data, and Control Hazards

Sequential Processing

Parallel Processing (Multi-core)

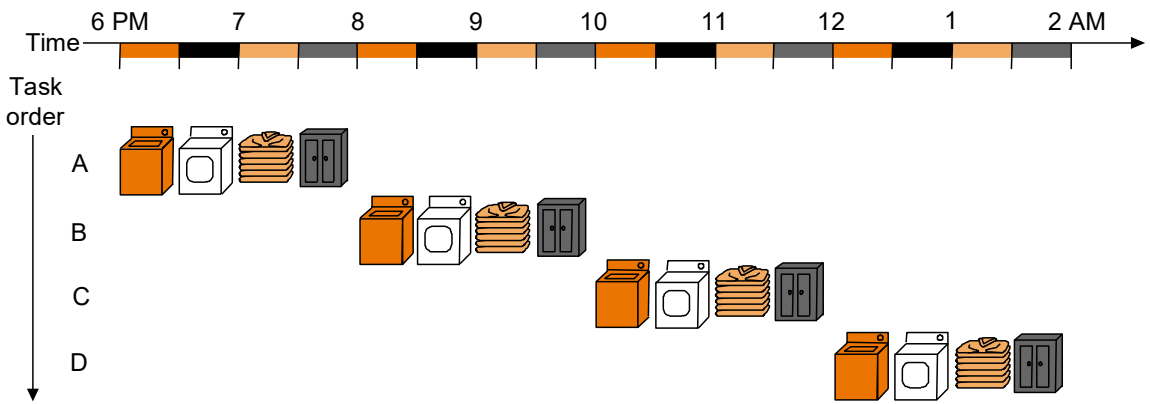
Pipelining

Laundry Example

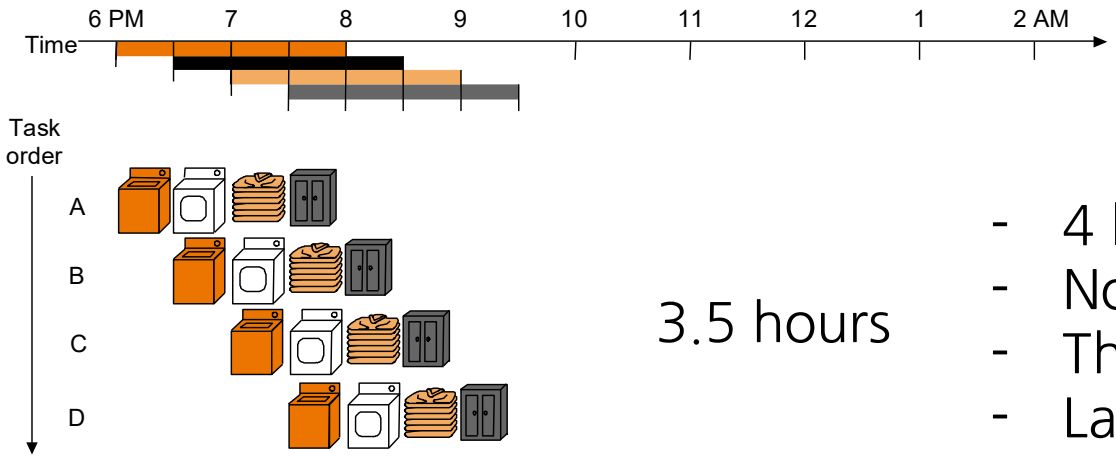


- Sequential laundry takes 8 hours for 4 loads
- But
  - Steps to do a load are sequentially dependent
  - No dependence between different loads
  - Different steps do not share resources

Pipelining Multiple Loads of Laundry



8 hours



3.5 hours

- 4 loads of laundry in parallel
- No additional resources
- Throughput increased by 4
- Latency per load is the same

Pipelining

- Important performance technique
  - Improves instruction throughput, not instruction latency
- Latency vs. throughput: two views of performance
  - (1) at the program level and (2) at the instruction level
- Single instruction latency
  - Doesn't matter: programs comprised of billions of instructions
  - Difficult to reduce anyway
- Goal is to make programs, not individual instructions, go faster
  - Instruction throughput → program latency
  - Key: exploit inter-instruction parallelism

RISC-V Pipeline

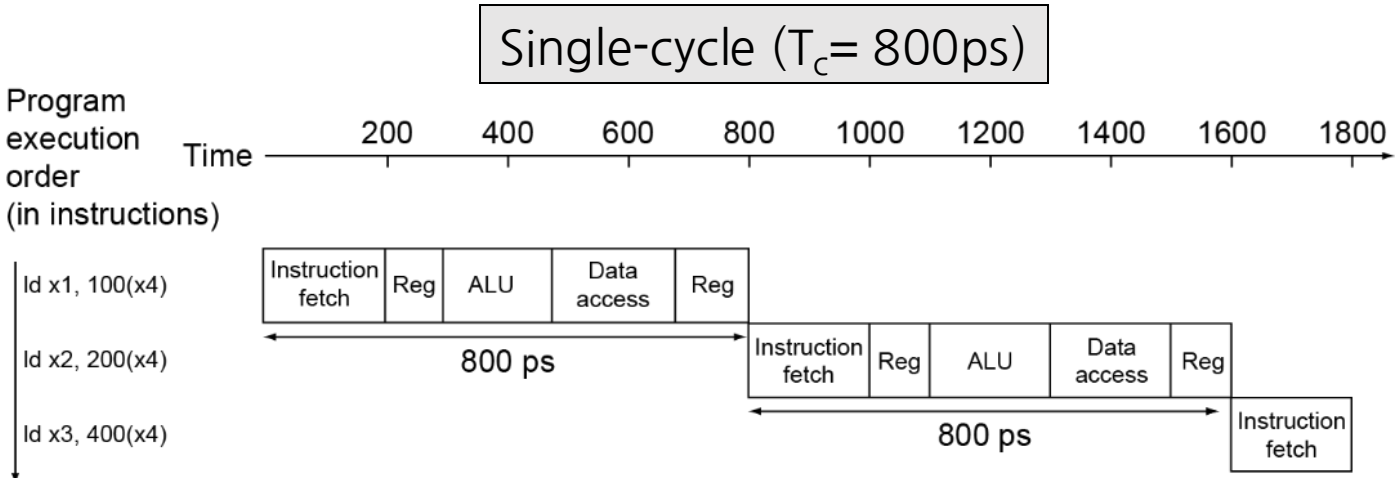
- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

Pipeline Performance

- Assume time for stages is
  - 100ps for register read or write
  - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
`ld`	200ps	100 ps	200ps	200ps	100 ps	800ps
`sd`	200ps	100 ps	200ps	200ps		700ps
`R-format`	200ps	100 ps	200ps		100 ps	600ps
`beq`	200ps	100 ps	200ps			500ps

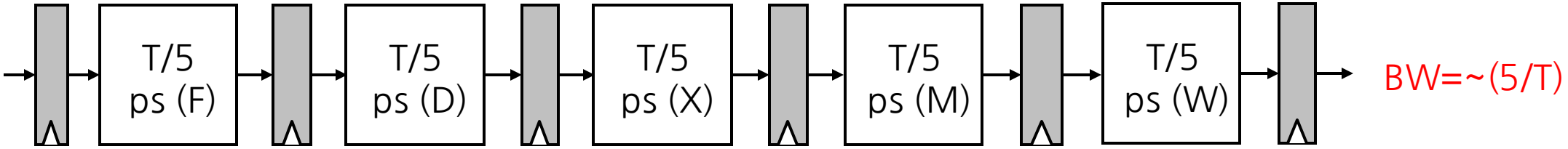
Pipeline Performance



Ideal Pipelining

- Non-pipelined version

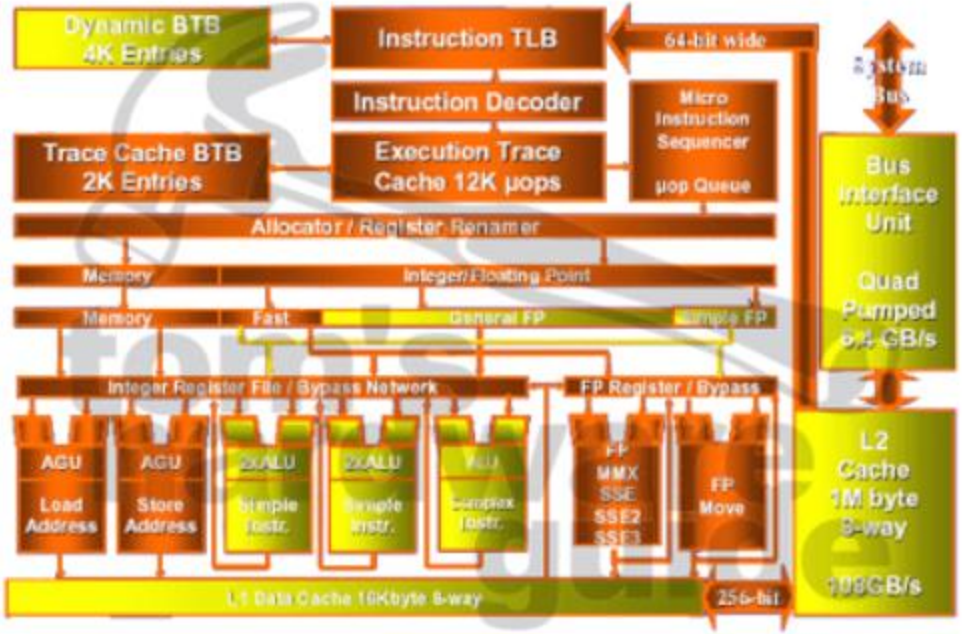
- 5-stage pipelined version



Intel's New Weapon: Pentium 4 Prescott

by Patrick Schmid February 1, 2004 at 7:00 PM

NetBurst Architecture: Now 31 Pipeline Stages



Throughput and Cost

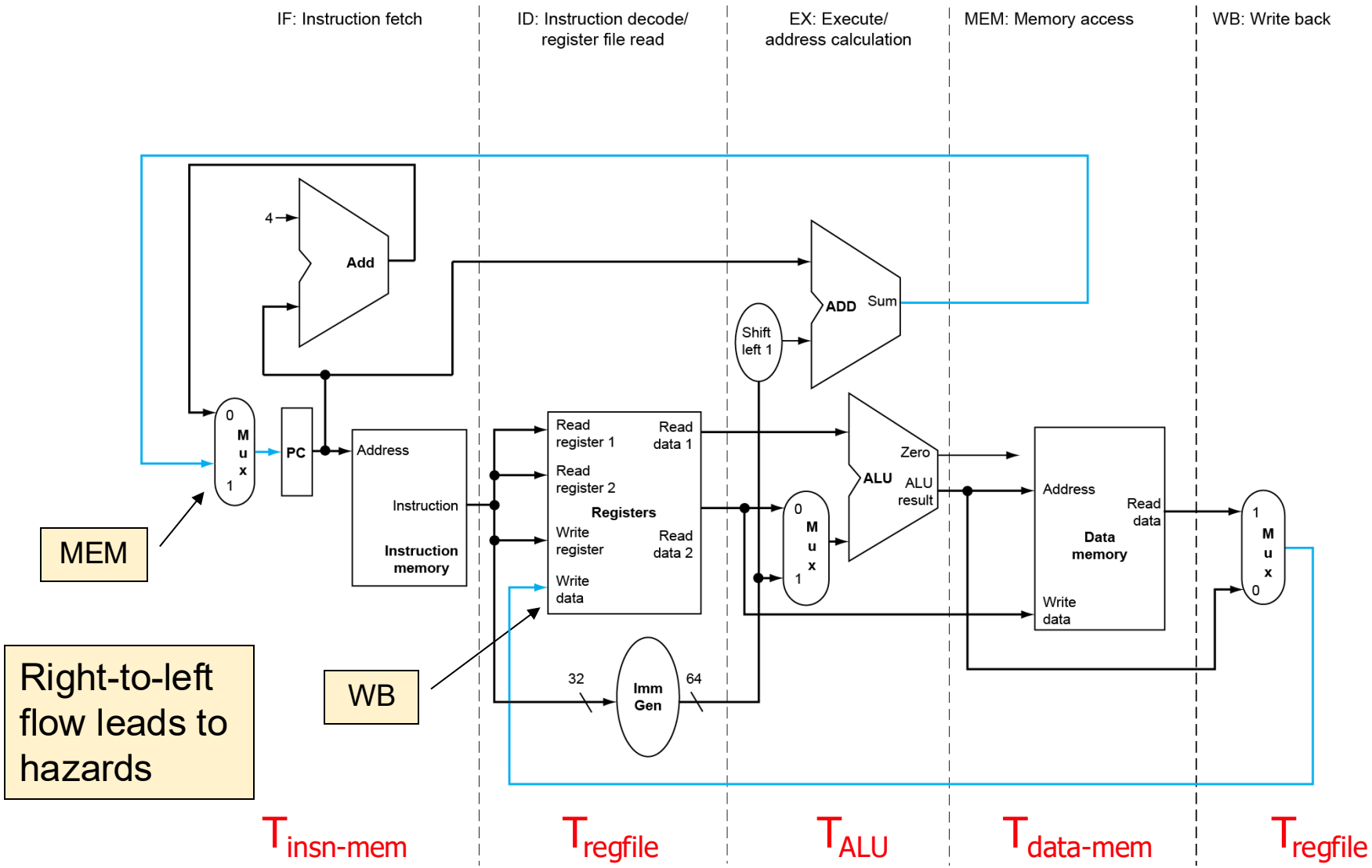
- Register delay
  - BW = 1/(T+S) where S = Register delay

- Register cost
  - Cost = G+L where L = Register cost

Pipelining and ISA Design

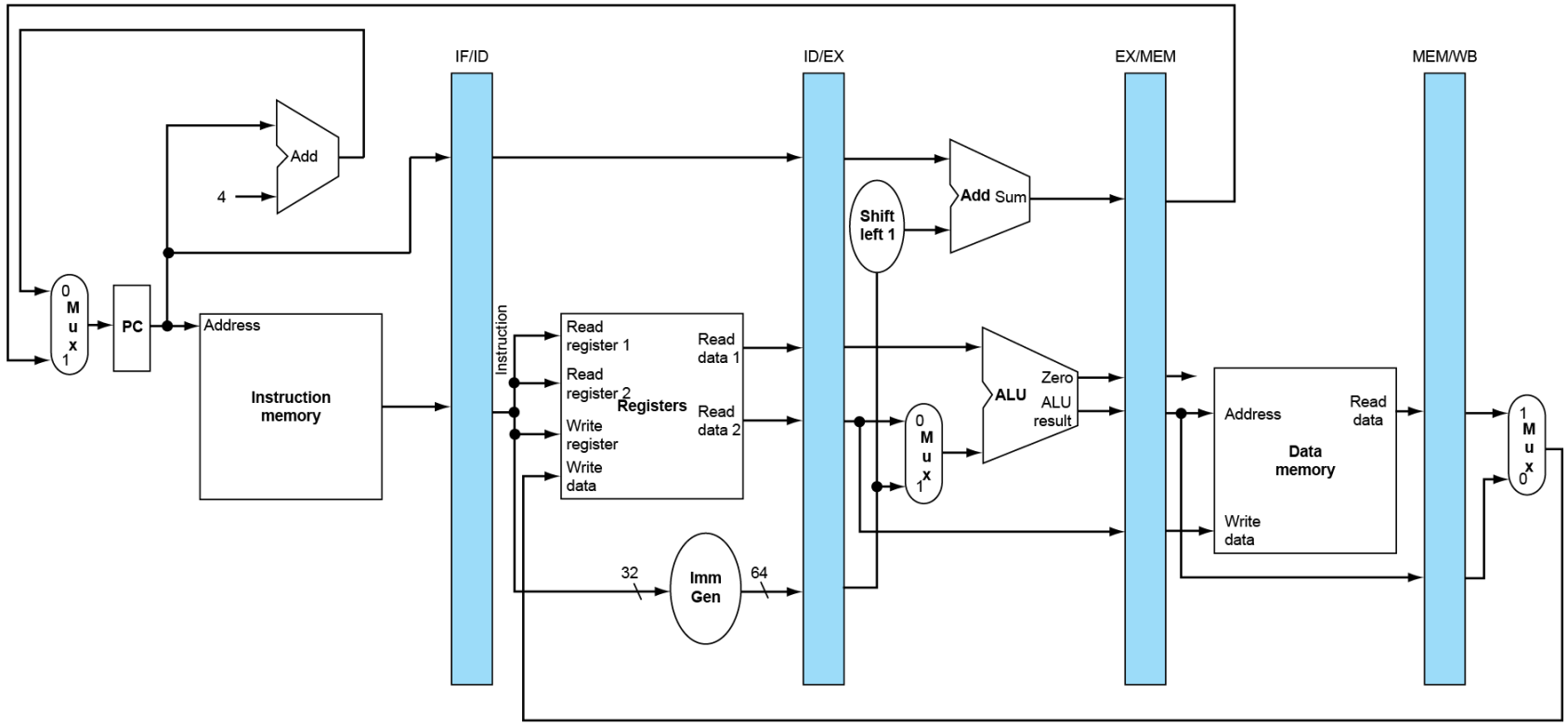
- RISC-V ISA designed for pipelining
  - All instructions are 32-bits
    - Easier to fetch and decode in one cycle
    - c.f. x86: 1- to 17-byte instructions
  - Few and regular instruction formats
    - Can decode and read registers in one step
  - Load/store addressing
    - Can calculate address in 3rd stage, access memory in 4th stage

RISC-V Pipelined Datapath



Pipeline registers

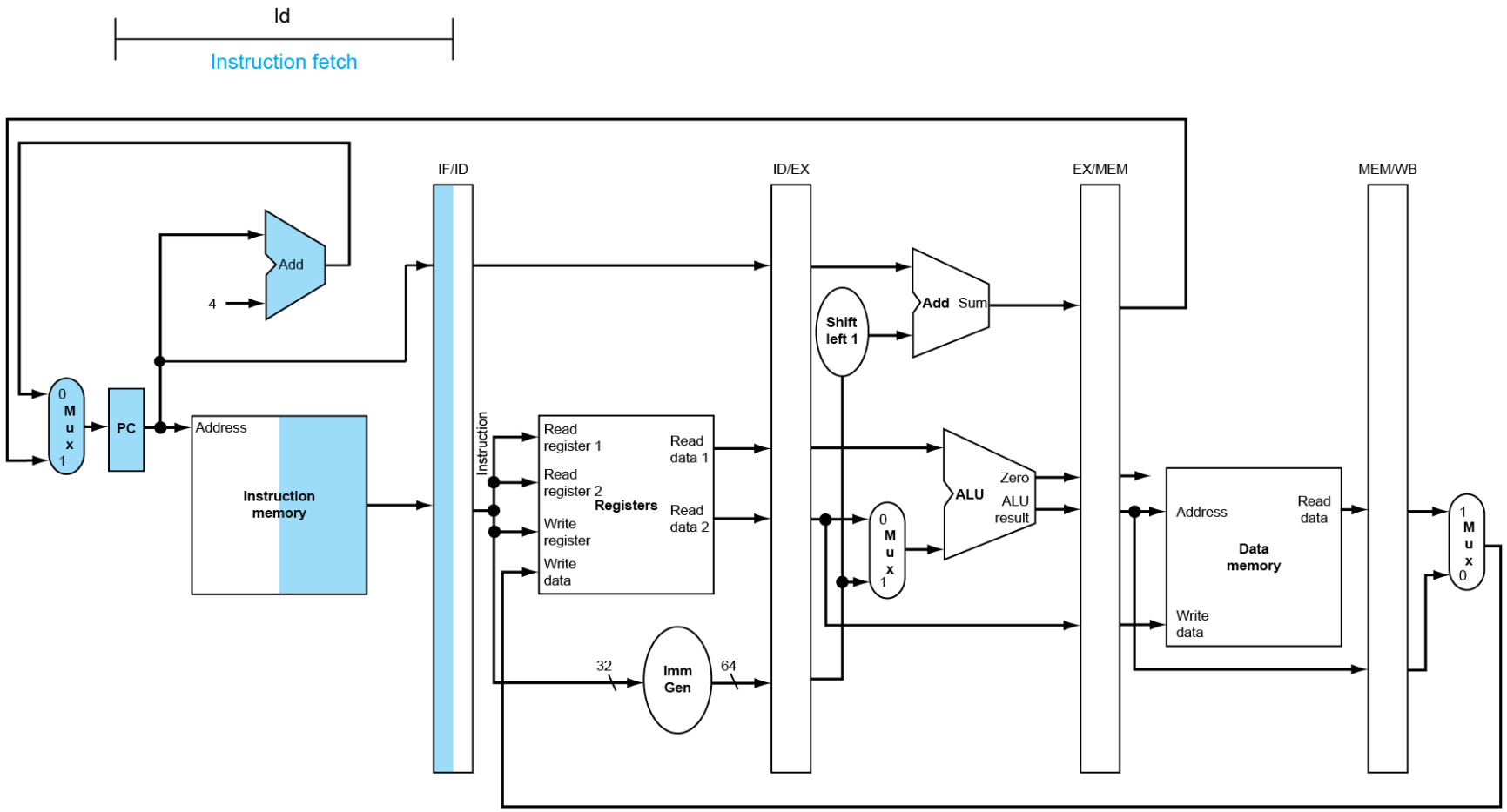
- Need registers between stages
  - To hold information produced in previous cycle



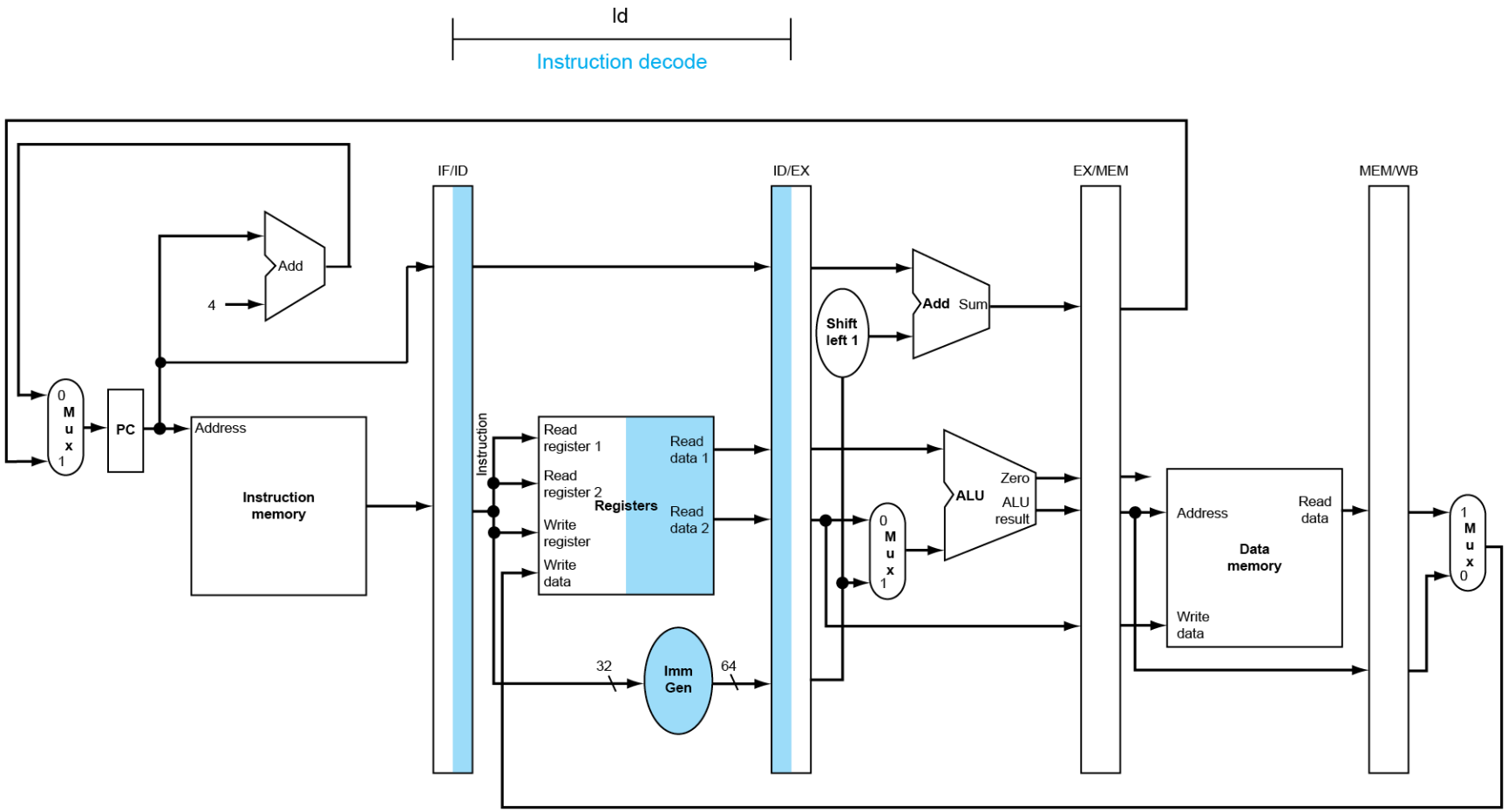
Pipeline Operation

- Cycle-by-cycle flow of instructions through the pipelined datapath
  - “Single-clock-cycle” pipeline diagram
    - Shows pipeline usage in a single cycle
    - Highlight resources used
  - c.f. “multi-clock-cycle” diagram
    - Graph of operation over time
- We’ll look at “single-clock-cycle” diagrams for load & store

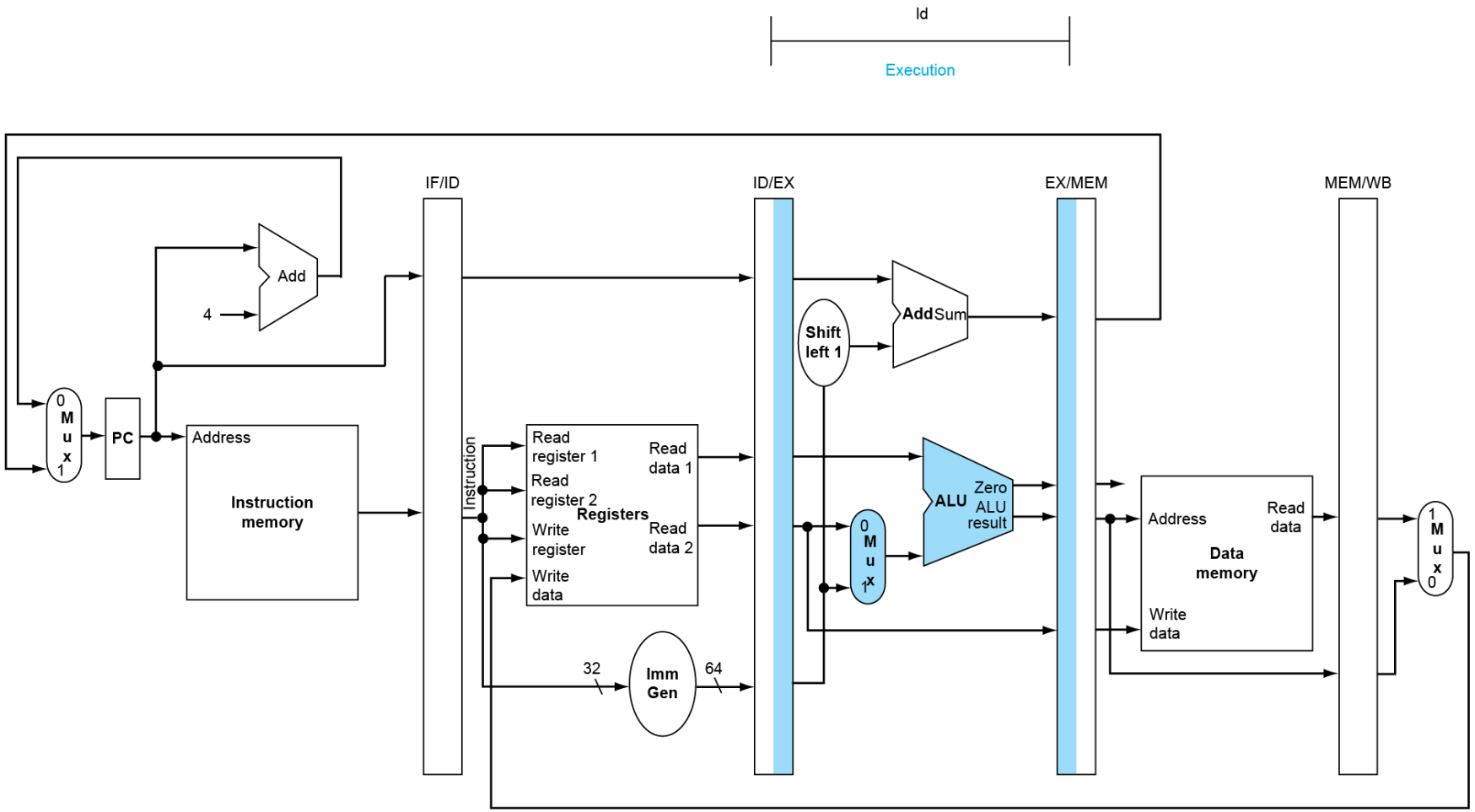
IF for Load, Store, …



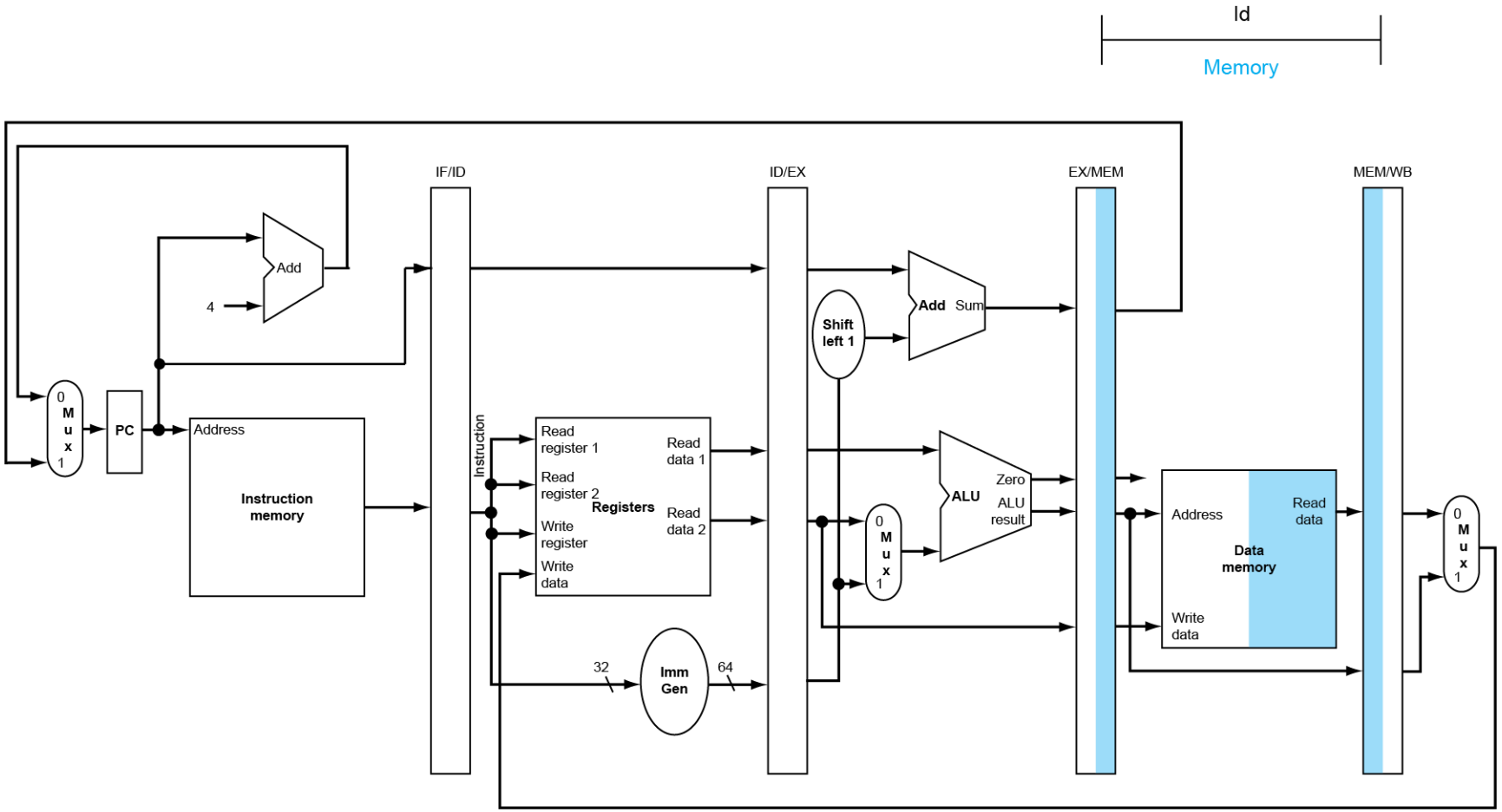
ID for Load, Store, …



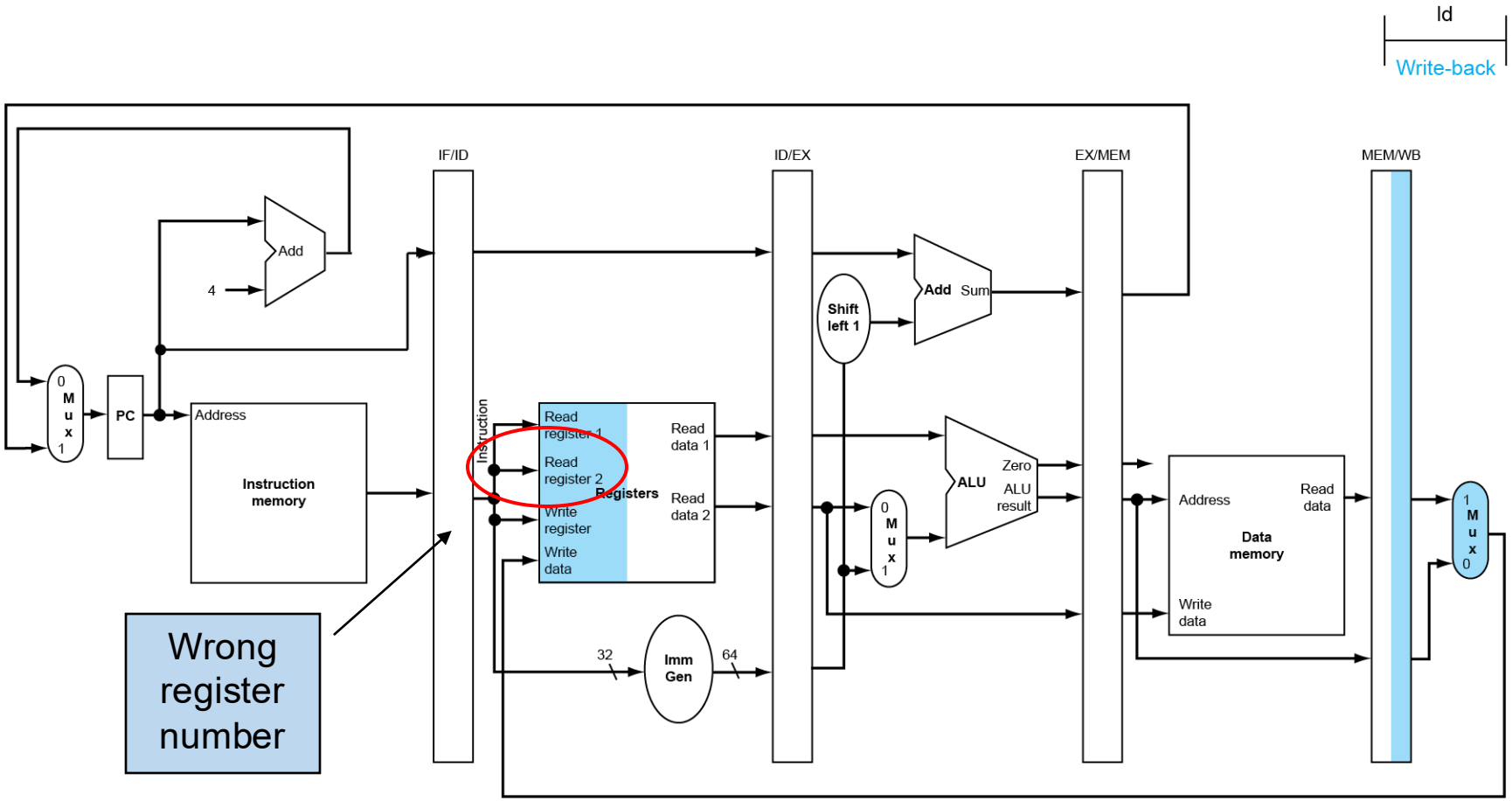
EX for Load


ld
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
32
Imm Gen
64
Shift left 1
Add
Sum
ALU
Zero
ALU result
Address
Data memory
Read data
Write data
1
M
u
x
0

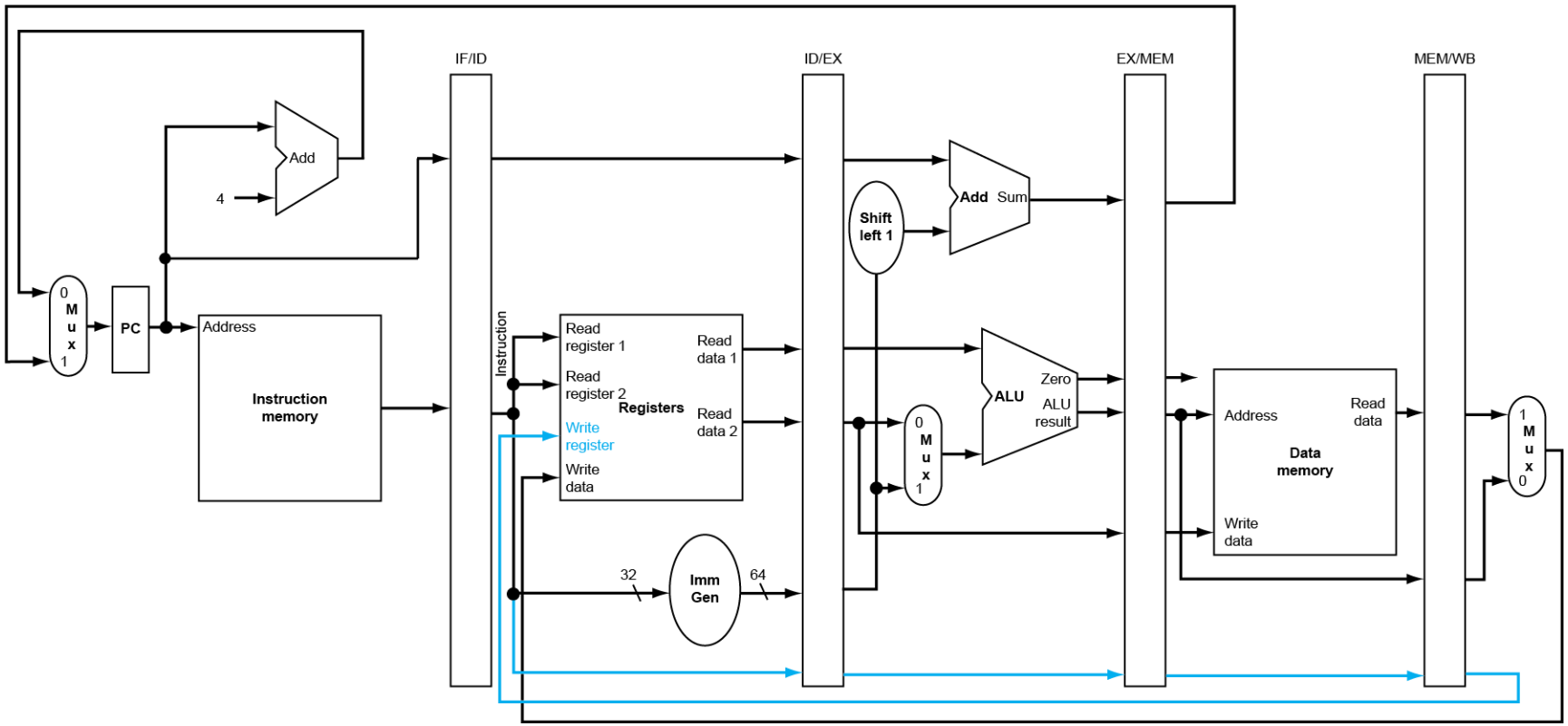
MEM for Load


ld
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
32
Imm Gen
64
Shift left 1
Add
Sum
ALU
Zero
ALU result
Address
Data memory
Read data
Write data

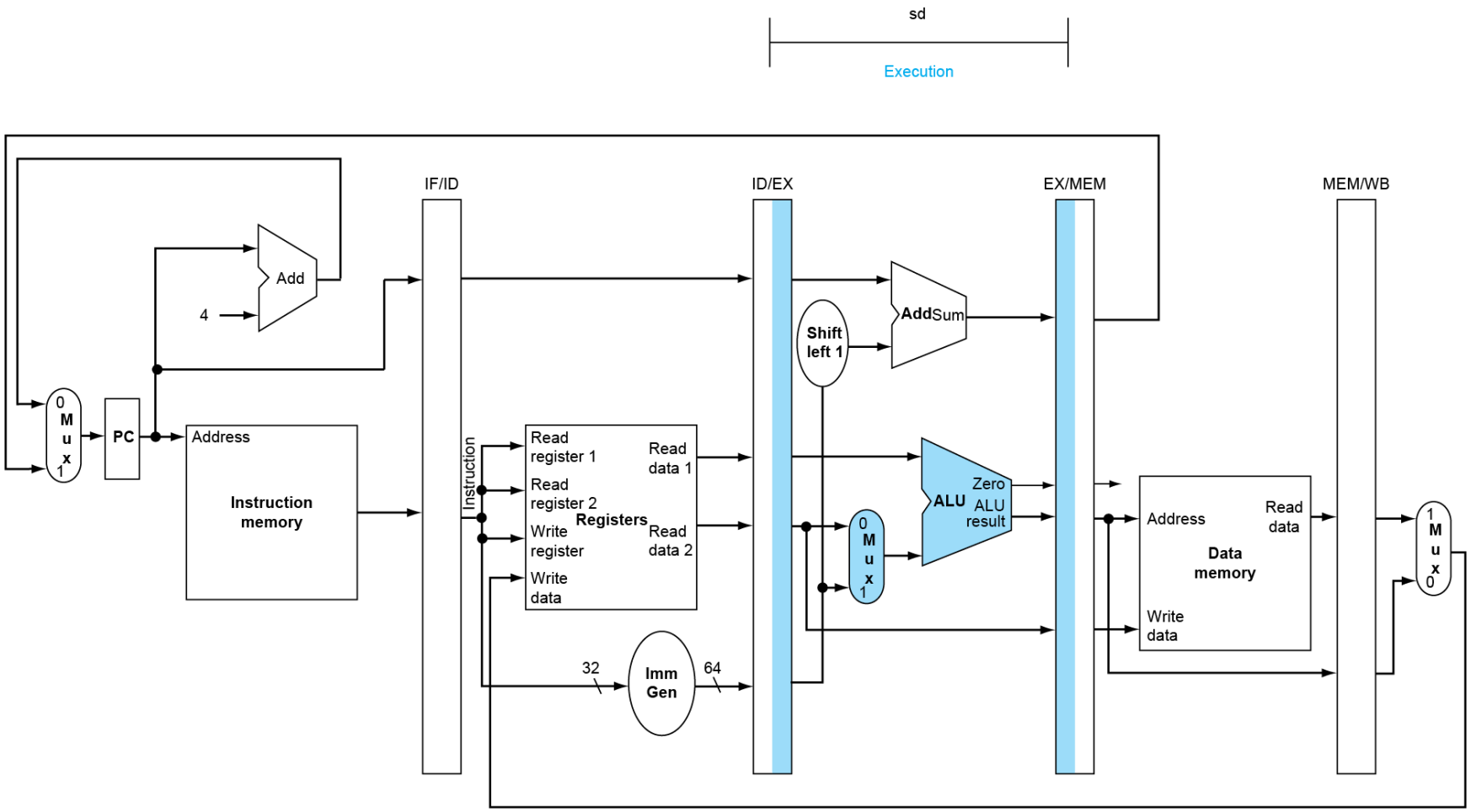
WB for Load


ld
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
32
Imm Gen
64
Shift left 1
Add
Sum
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
1
M
u
x
0
Wrong register number

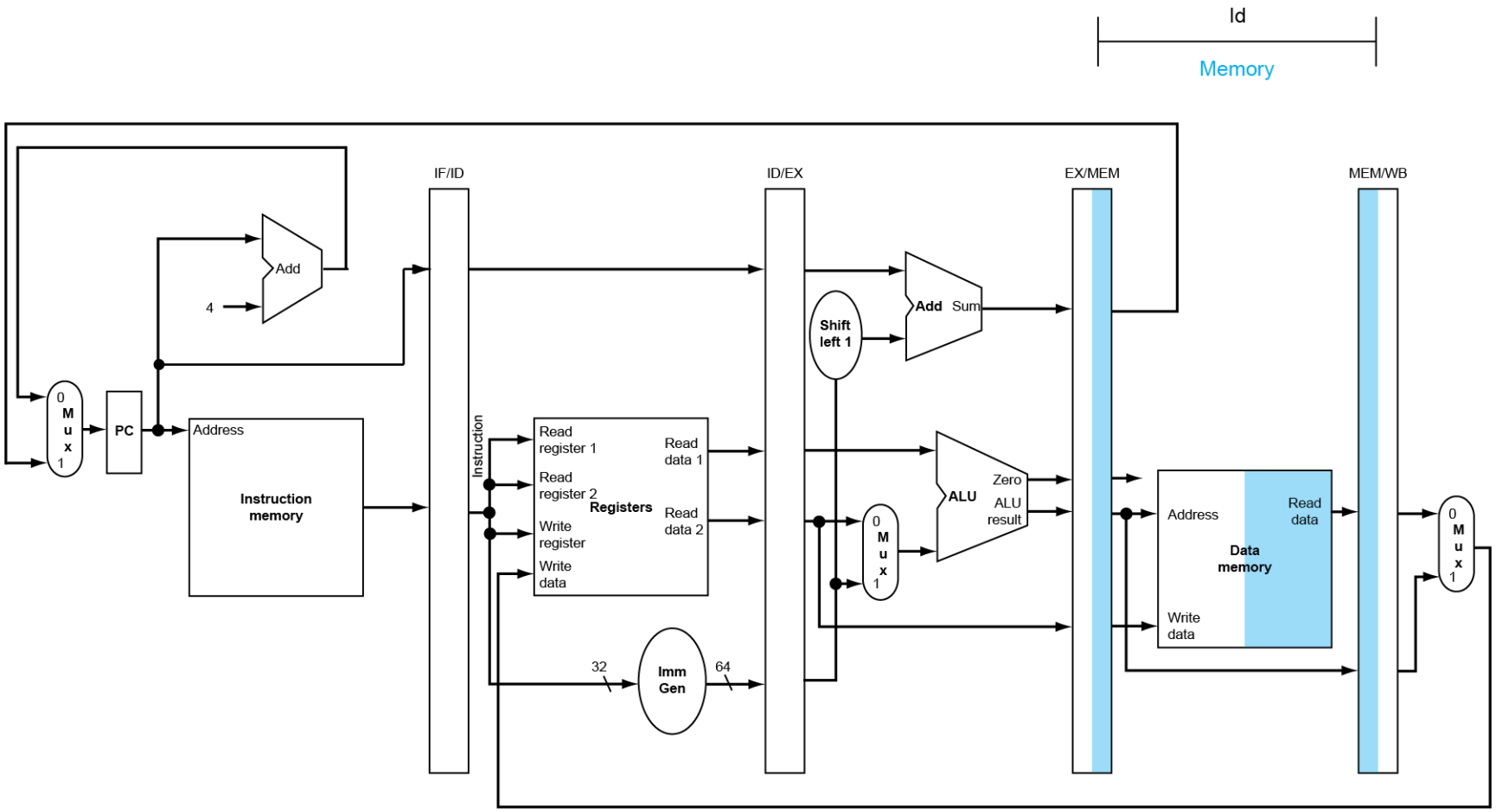
Corrected Datapath for Load



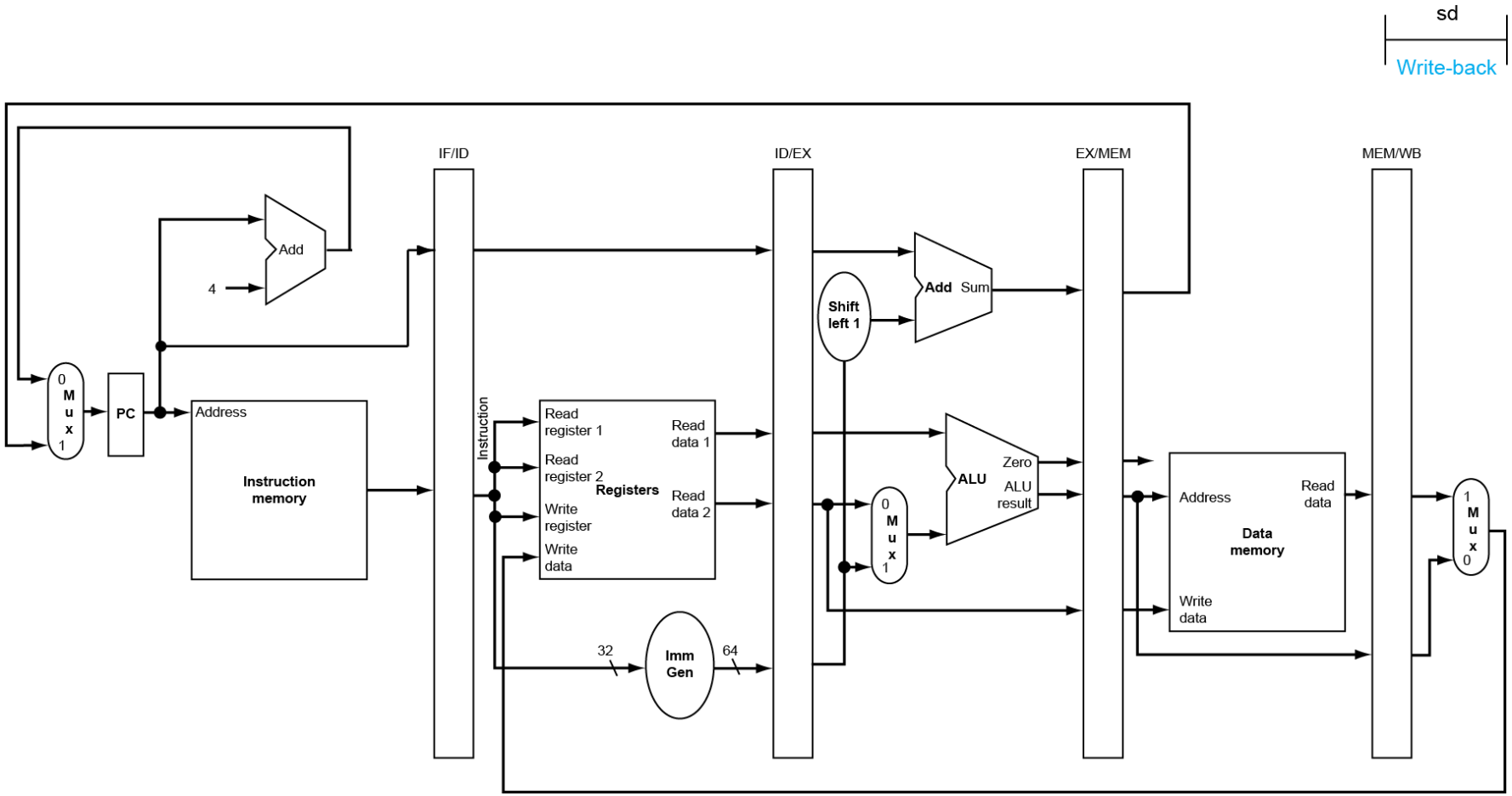
EX for Store


sd
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Write register
Write data
Registers
Read data 1
Read data 2
32
Imm Gen
64
Shift left 1
AddSum
ALU
Zero
ALU result
0
M
u
x
1
Address
Data memory
Read data
Write data
1
M
u
x
0

MEM for Store

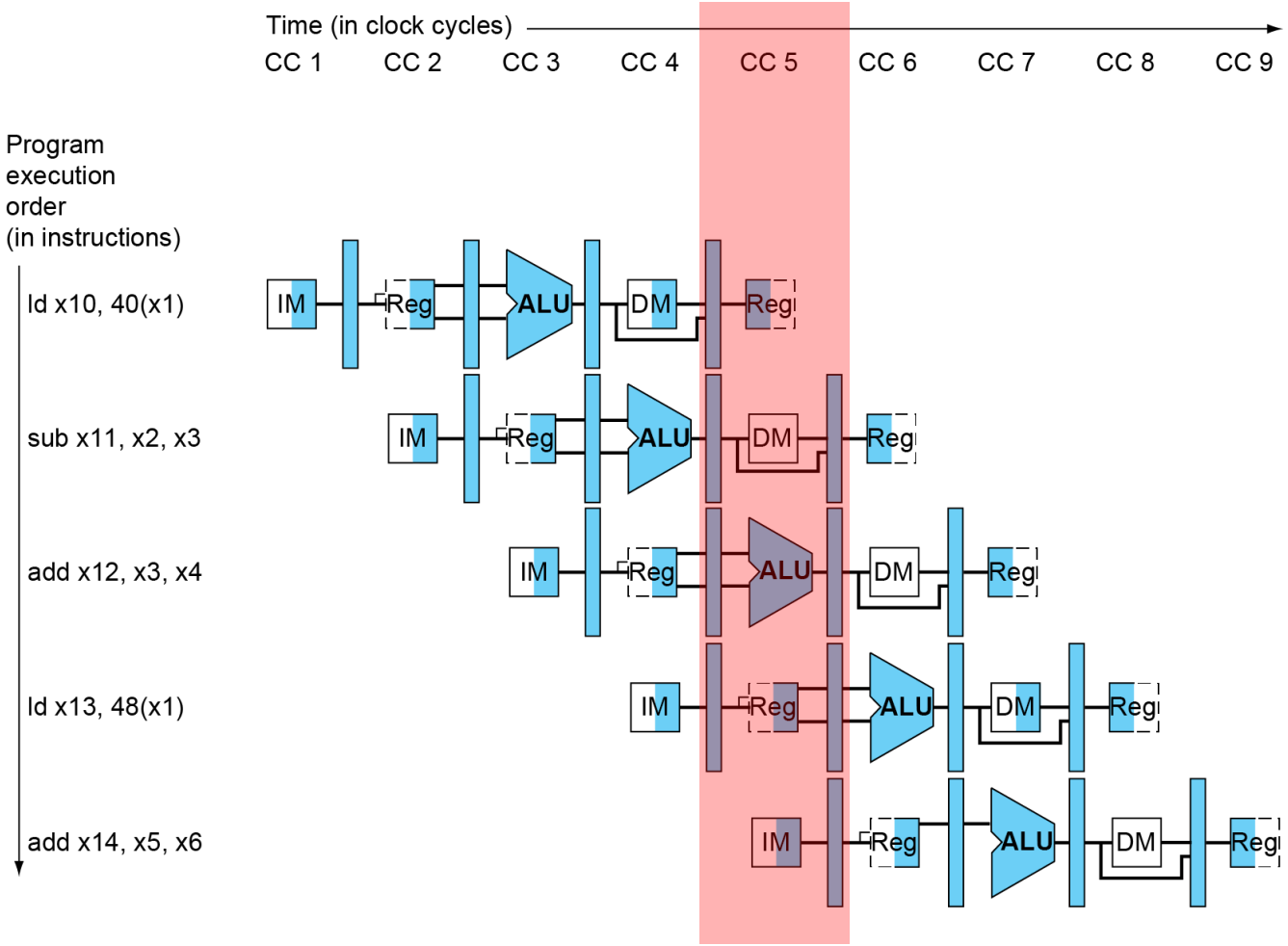


WB for Store


sd
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
32
Imm Gen
64
Shift left 1
Add
Sum
ALU
Zero
ALU result
0
M
u
x
1
Address
Read data
Data memory
Write data
1
M
u
x
0

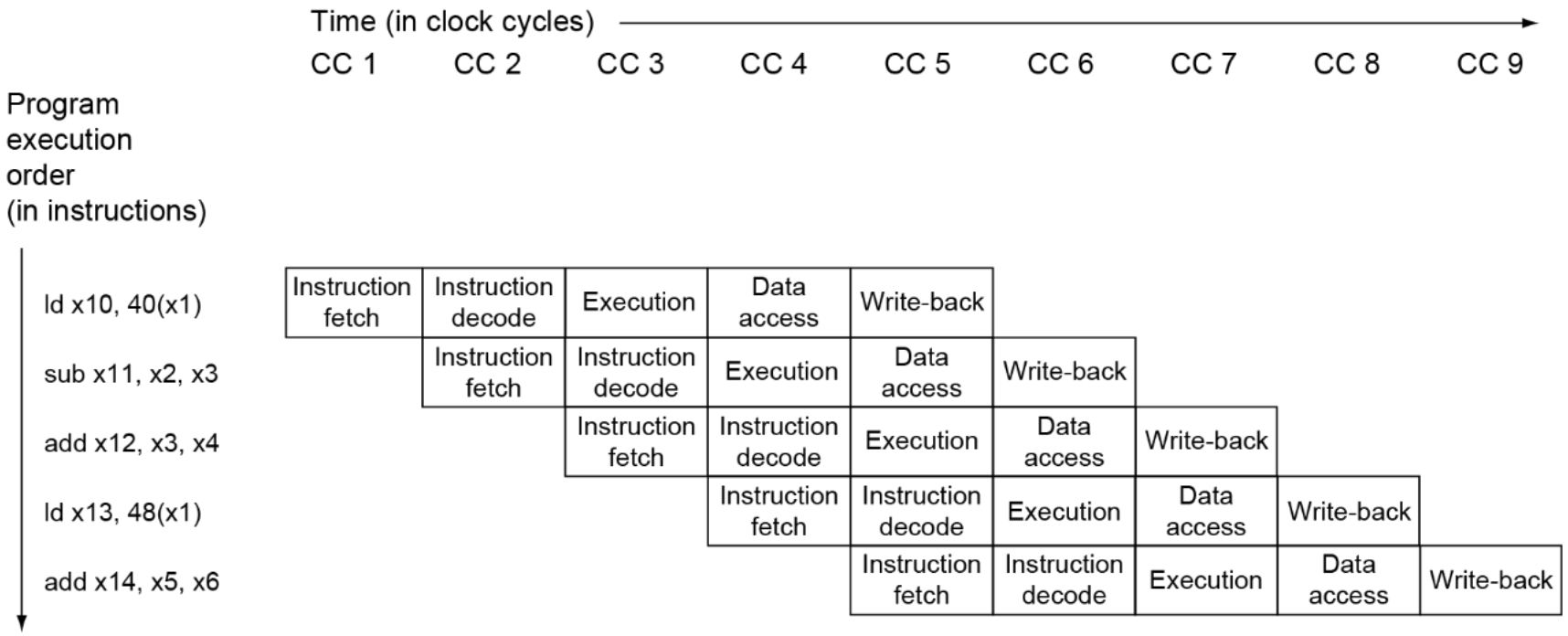
Multi-Cycle Pipeline Diagram

- Form showing resource usage



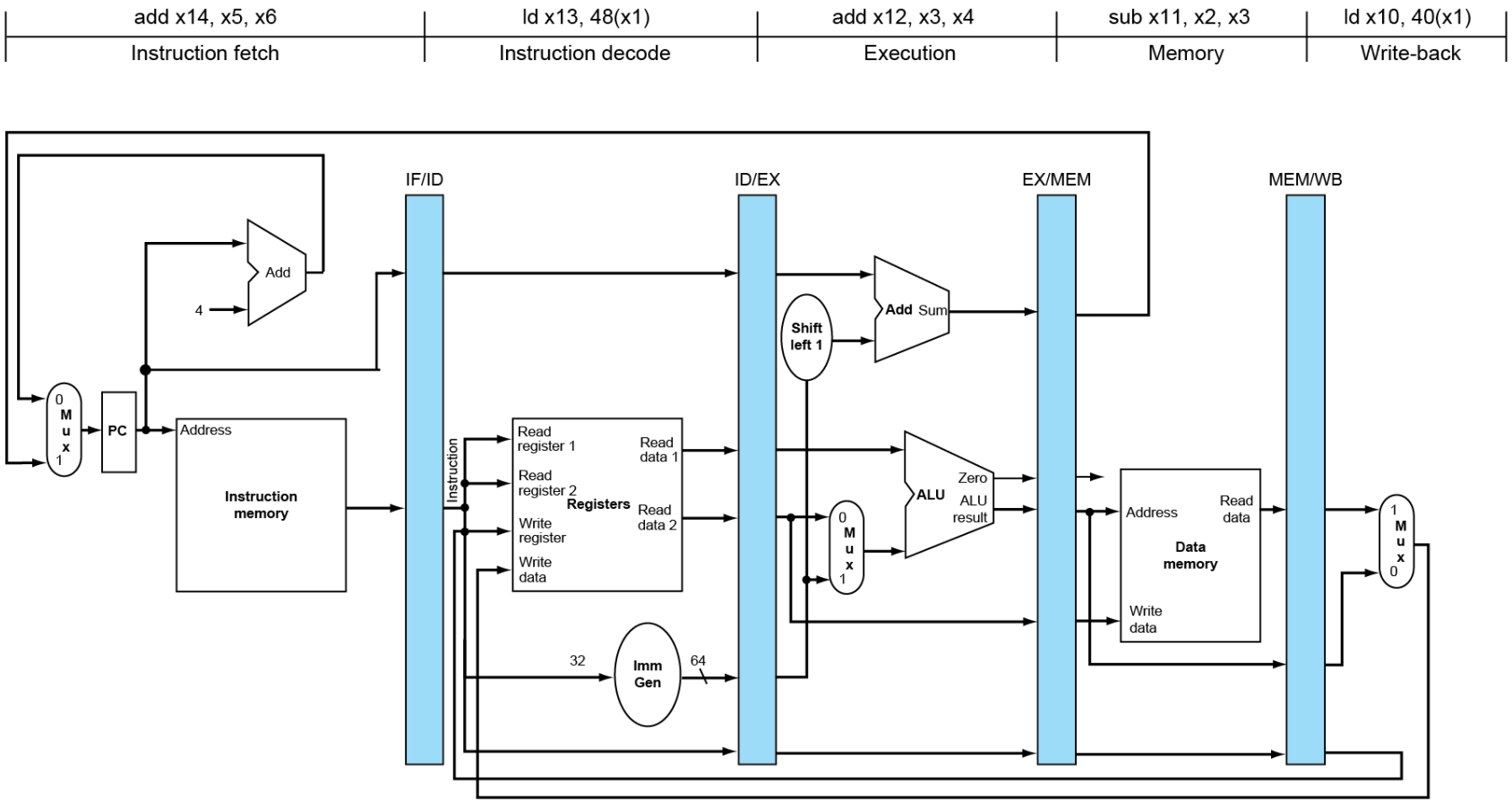
Multi-Cycle Pipeline Diagram

- Traditional form

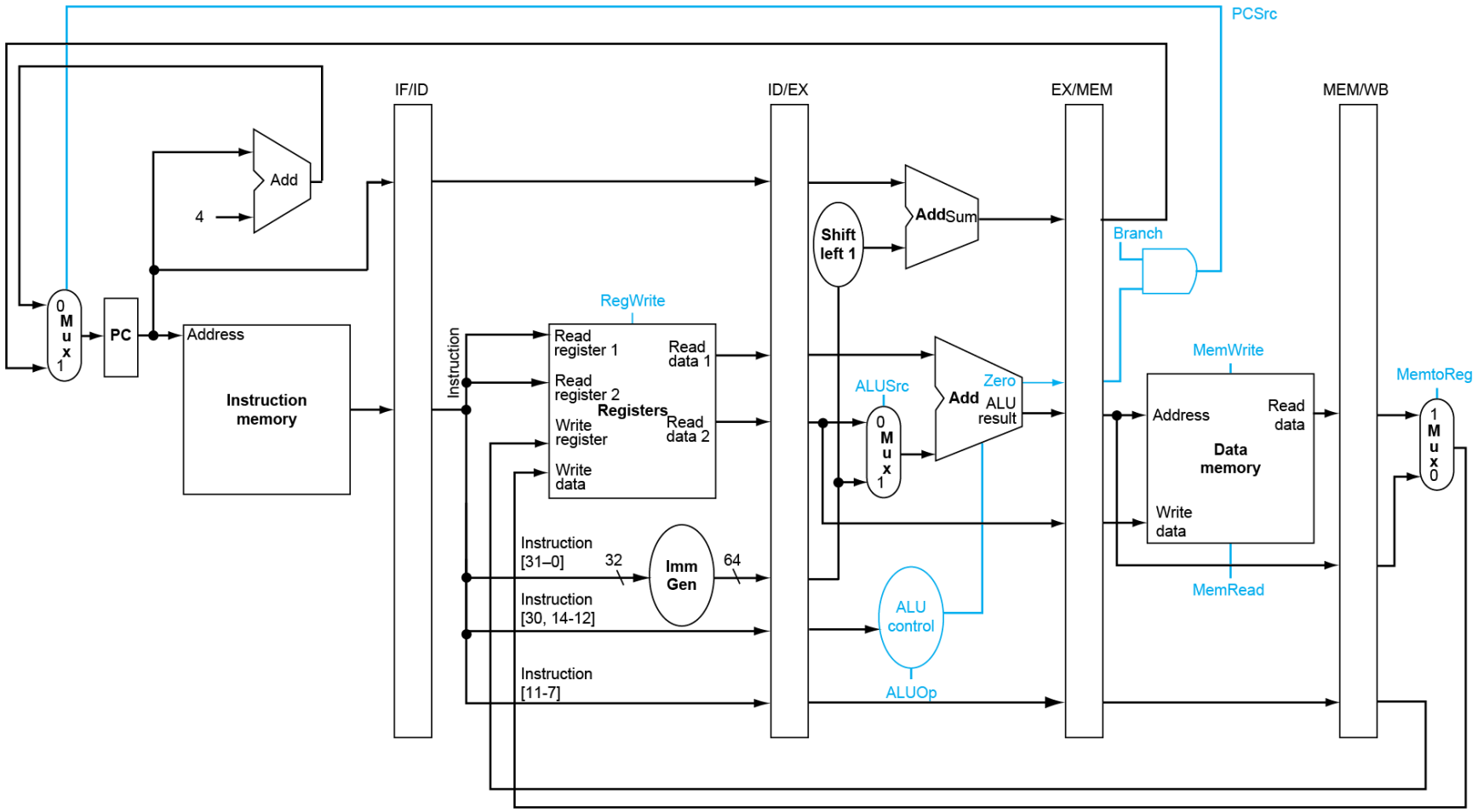


Single-Cycle Pipeline Diagram

- State of pipeline in a given cycle



Pipelined Control (Simplified)

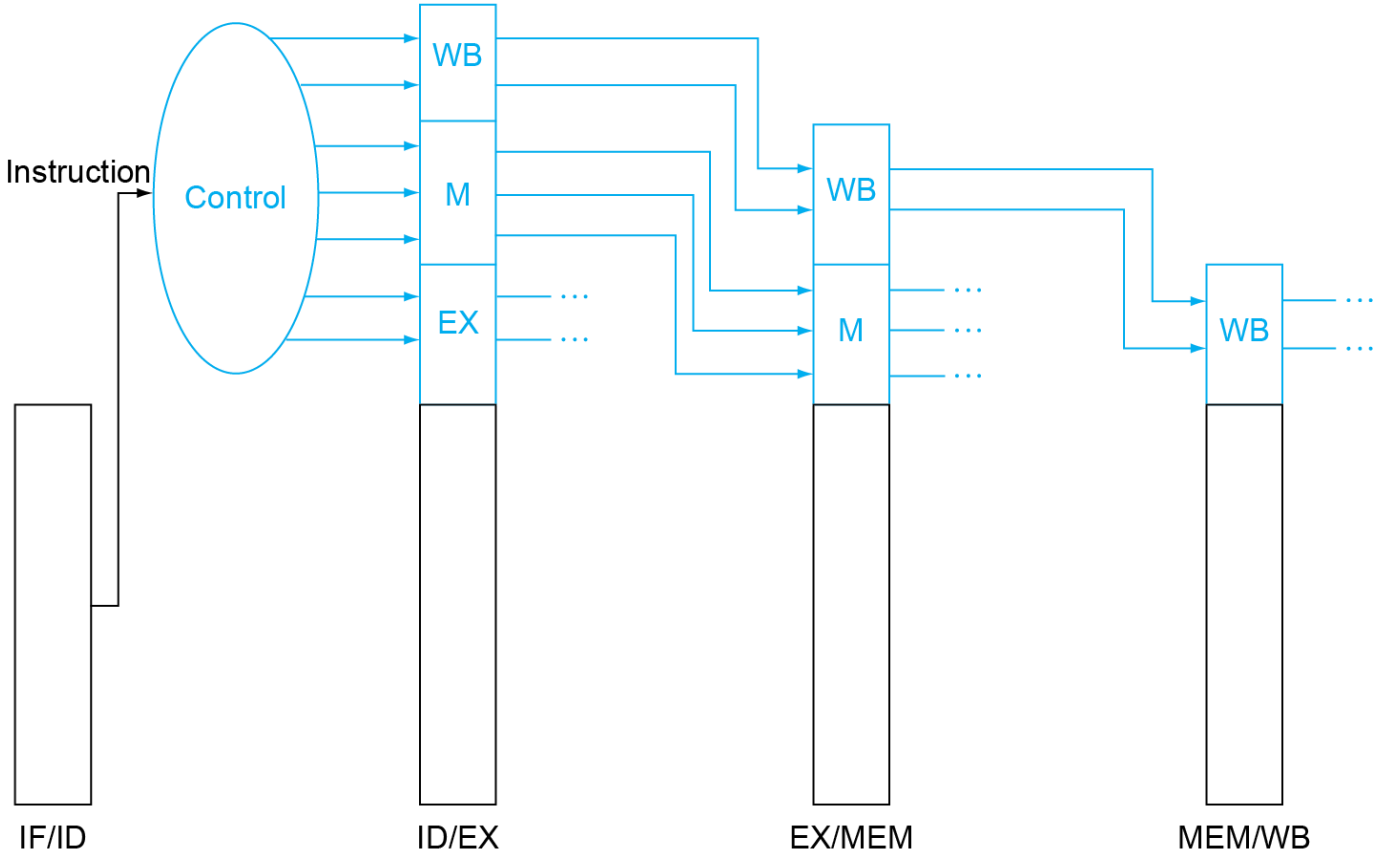


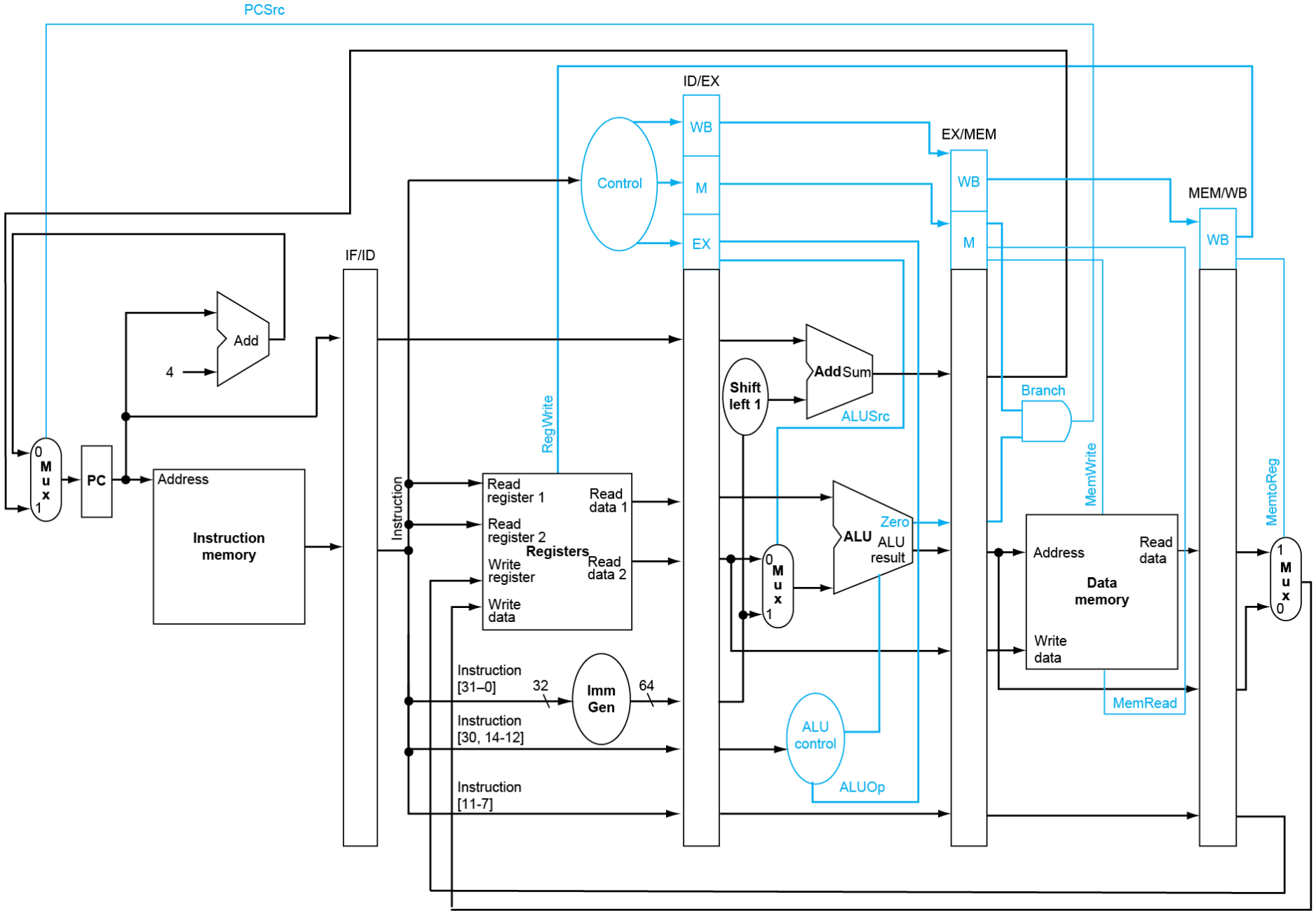
Generating Control Signals

Instruction	Execution/address calculation stage control lines		Memory access stage control lines			Write-back stage control lines	
	ALUOp	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R-format	10	0	0	0	0	1	0
ld	00	1	0	1	0	1	1
sd	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

Pipelined Control

- Control signals derived from instruction
  - As in single-cycle implementation




PCSrc
ID/EX
EX/MEM
MEM/WB
IF/ID
Control
WB
M
EX
WB
M
WB
Add
4
0
M
u
x
1
PC
Address
Instruction
memory
Instruction
RegWrite
Read
register 1
Read
register 2
Registers
Write
register
Write
data
Read
data 1
Read
data 2
Instruction
[31–0]
32
Imm
Gen
64
Instruction
[30, 14-12]
Instruction
[11-7]
Shift
left 1
Add
Sum
ALUSrc
0
M
u
x
1
ALU
Zero
ALU
result
ALU
control
ALUOp
Branch
MemWrite
Address
Read
data
Data
memory
Write
data
MemRead
MemtoReg
1
M
u
x
0

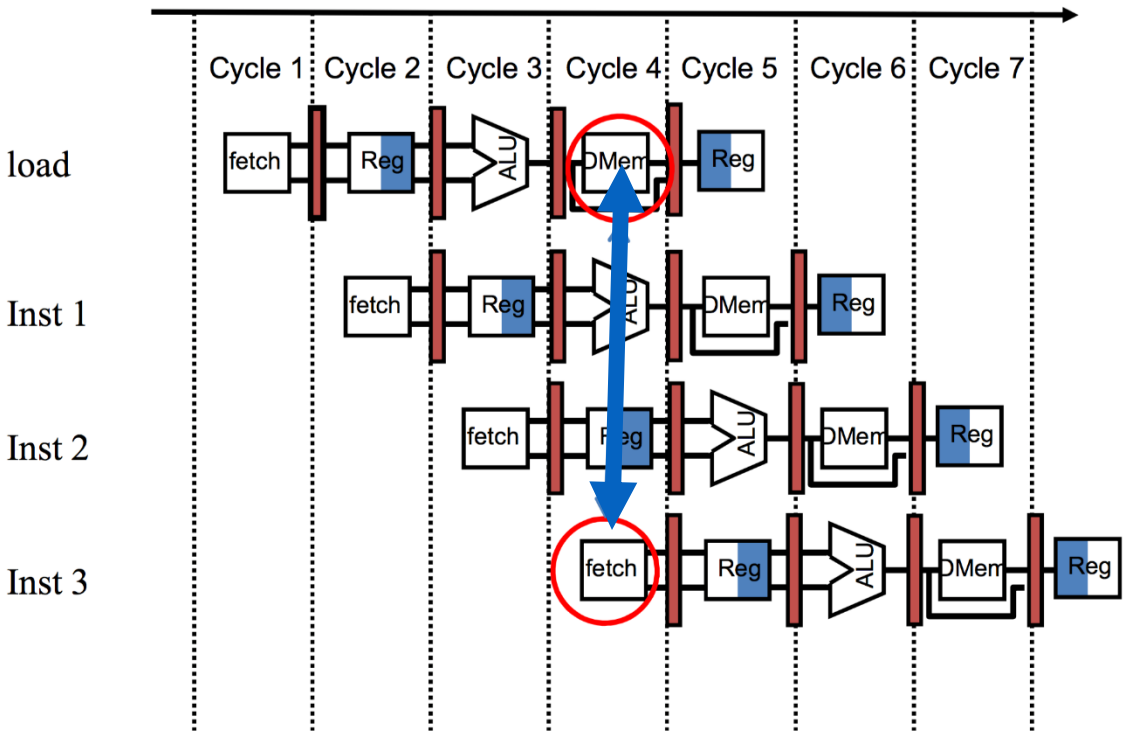
Pipeline Hazards

- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
  - A required resource is busy
- Data hazards
  - Need to wait for previous instruction to complete its data read/write
- Control hazards
  - Deciding on control action depends on previous instruction

Structure Hazards

- Conflict for use of a resource
- In RISC-V pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to **`stall`** for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches

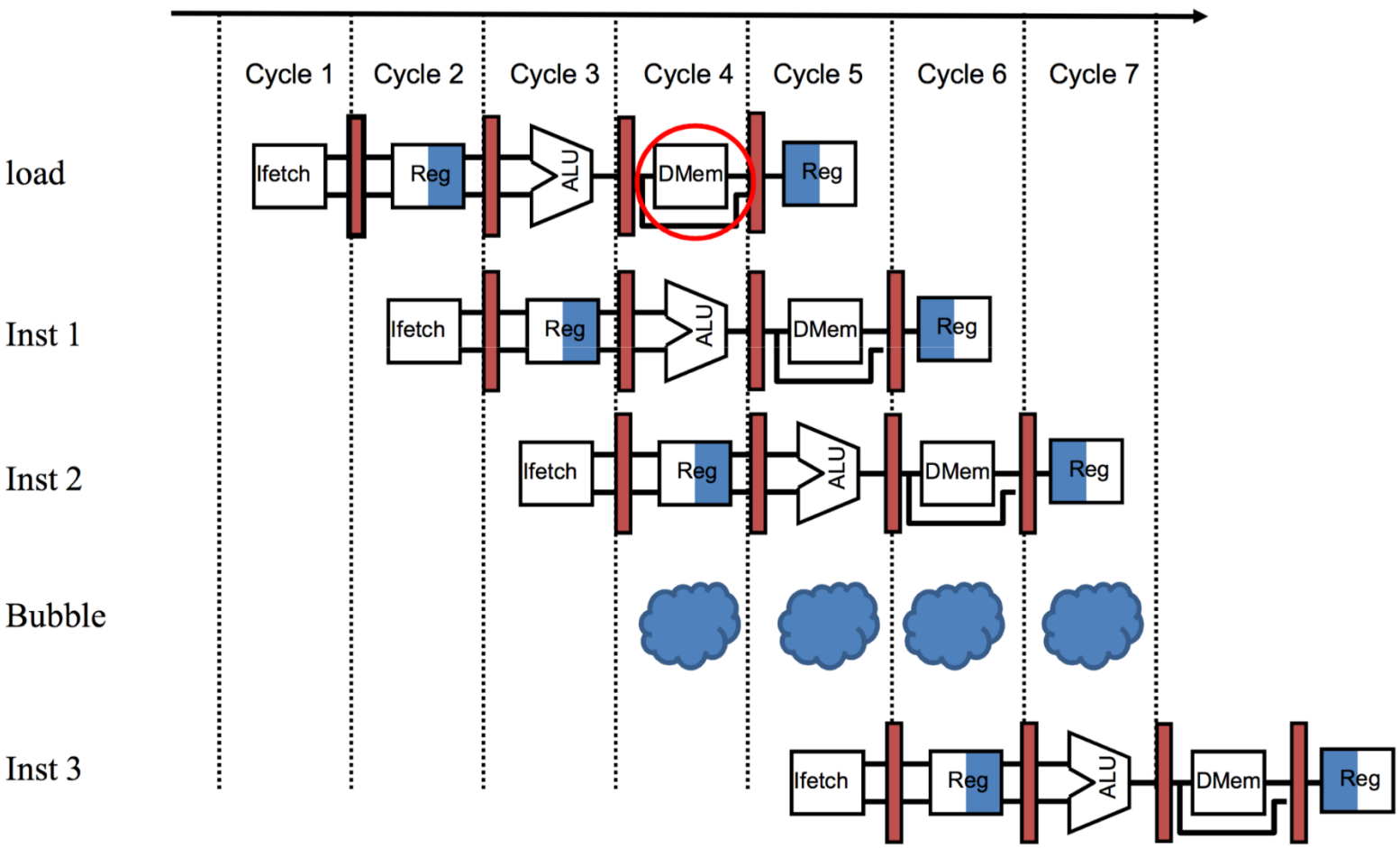
Example: Structural Hazard



- Suppose: single-ported memory for both instruction and data
- Solution: stall pipeline or add one more port

Example: Structural Hazard

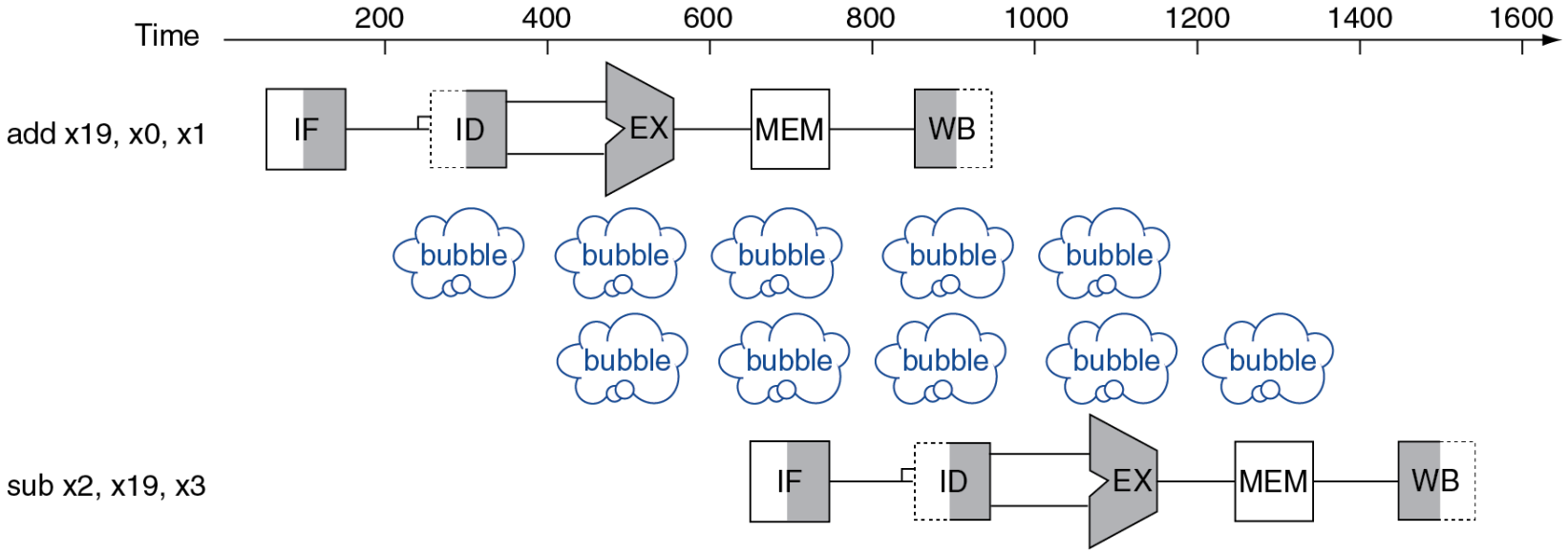
- Stalling pipe to avoid conflict access to memory



Data Hazards

- An instruction depends on completion of data access by a previous instruction

```
add   x19, x0, x1
sub   x2, x19, x3 // x19 has RAW Hazard (Read after write)
```



Data Hazards in ALU Instructions

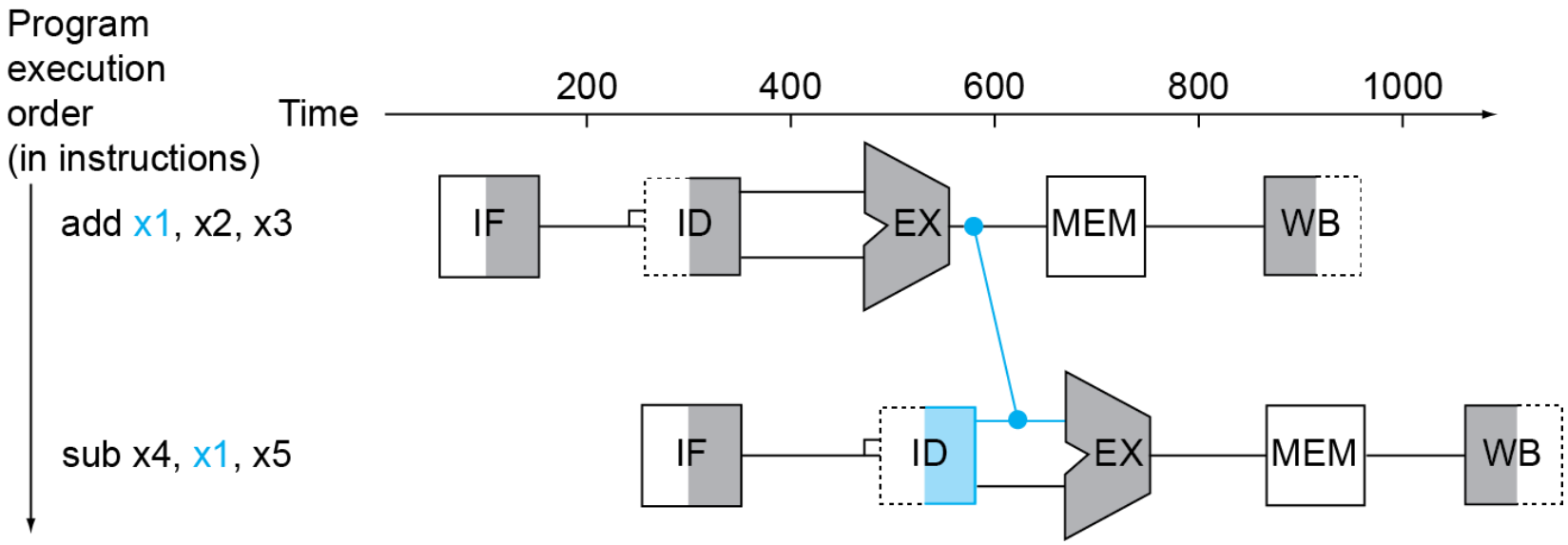
- Consider this sequence:

```
sub   x2, x1,x3
and   x12,x2,x5
or    x13,x6,x2
add   x14,x2,x2
sd    x15,100(x2)
```

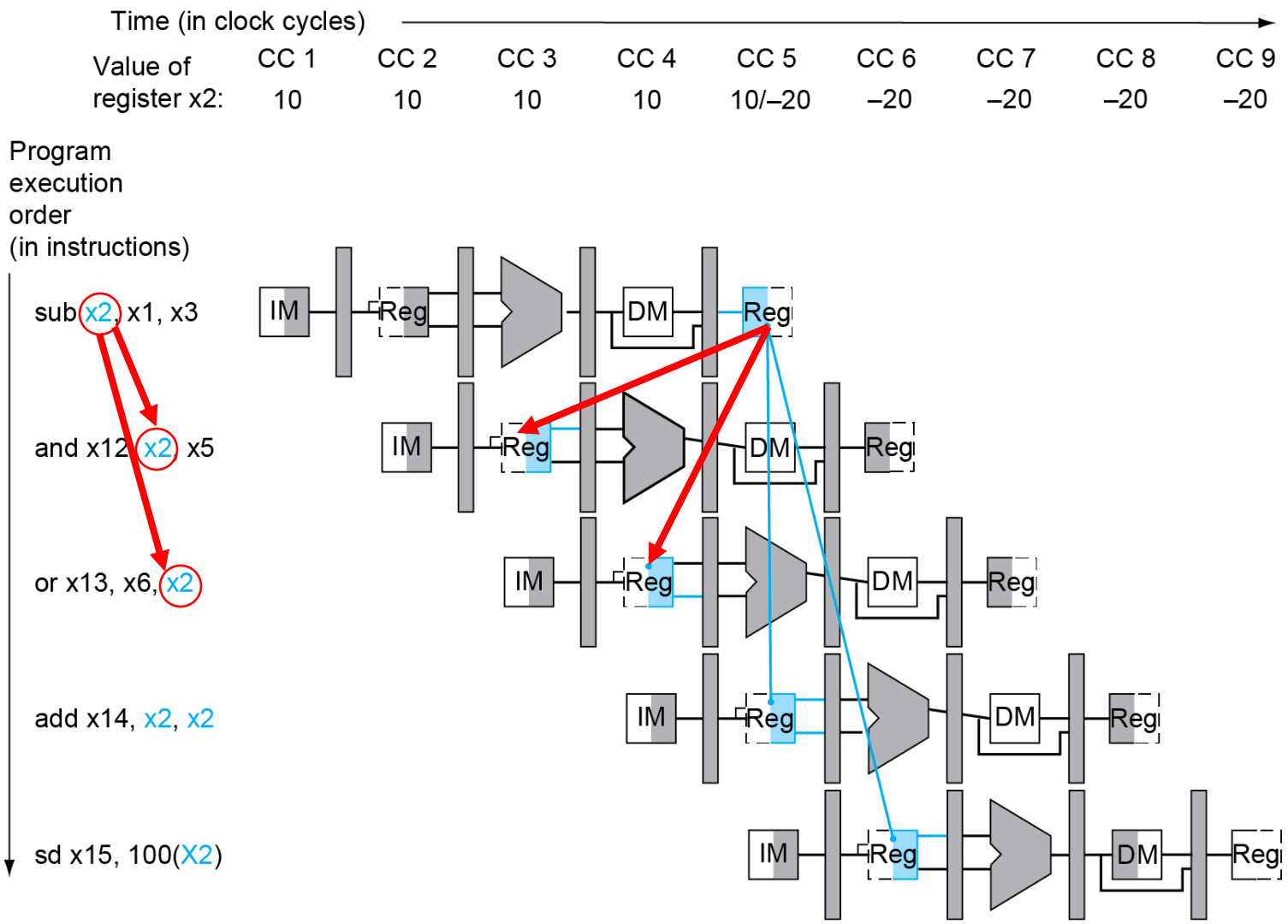
- We can resolve hazards with forwarding
  - How do we detect when to forward?

Forwarding (a.k.a Bypassing)

- Use result when it is computed
  - Don't wait for it to be stored in a register
  - Requires extra connections in the datapath

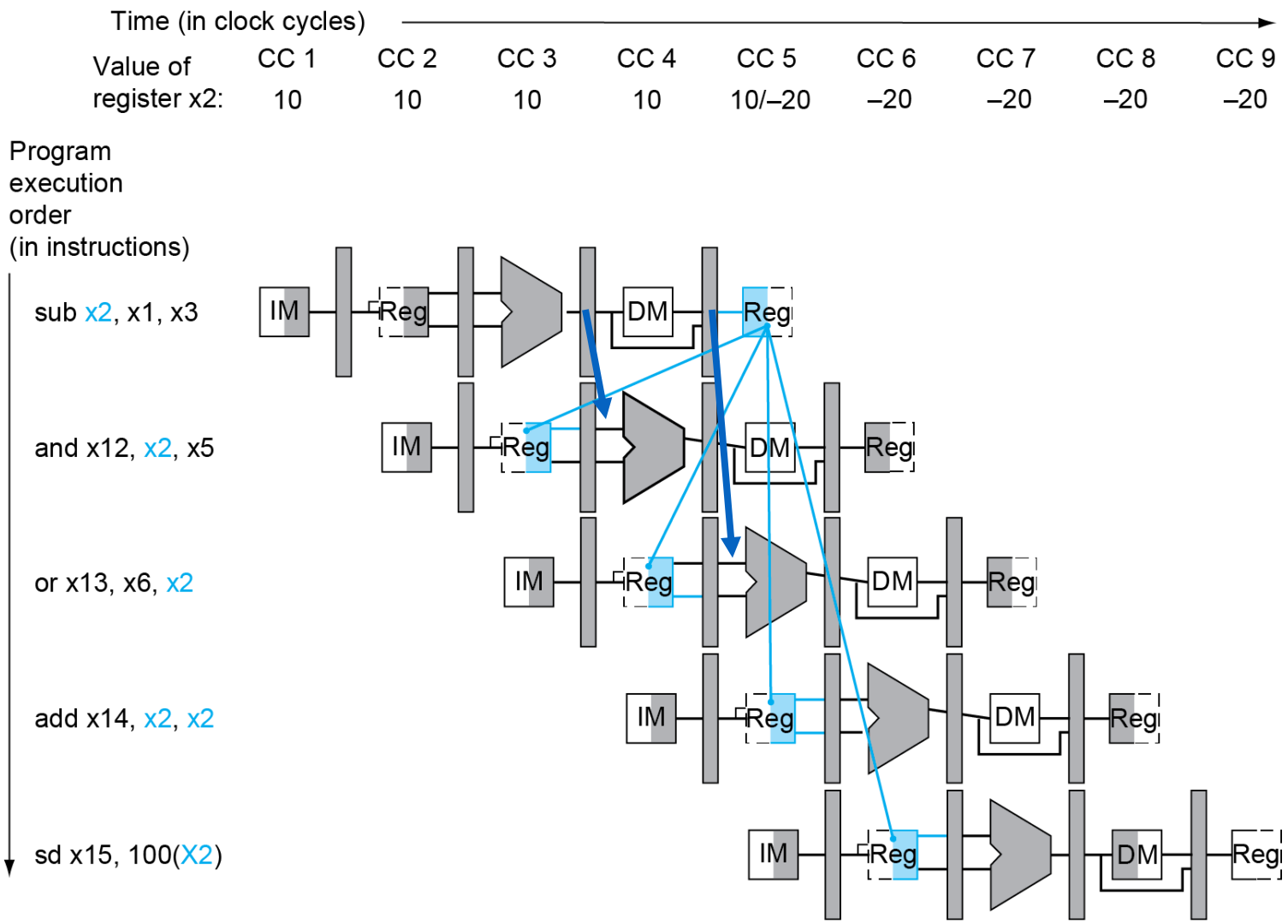


Data Hazard: RAW (Read After Write)



An instruction depends on completion of data access by previous instruction

Dependencies & Forwarding



The result of `sub` instruction can be known at the EX/MEM stage and MEM/WB stage

Detecting the Need to Forward

- Pass register numbers along pipeline
  - e.g., `ID/EX.RegisterRs1 = register #` for Rs1 sitting in `ID/EX` pipeline register
- ALU operand register numbers in `EX` stage are given by
  - `ID/EX.RegisterRs1, ID/EX.RegisterRs2`
- Data hazards when

```
EX/MEM.RegisterRd = ID/EX.RegisterRs1
EX/MEM.RegisterRd = ID/EX.RegisterRs2

MEM/WB.RegisterRd = ID/EX.RegisterRs1
MEM/WB.RegisterRd = ID/EX.RegisterRs2
```

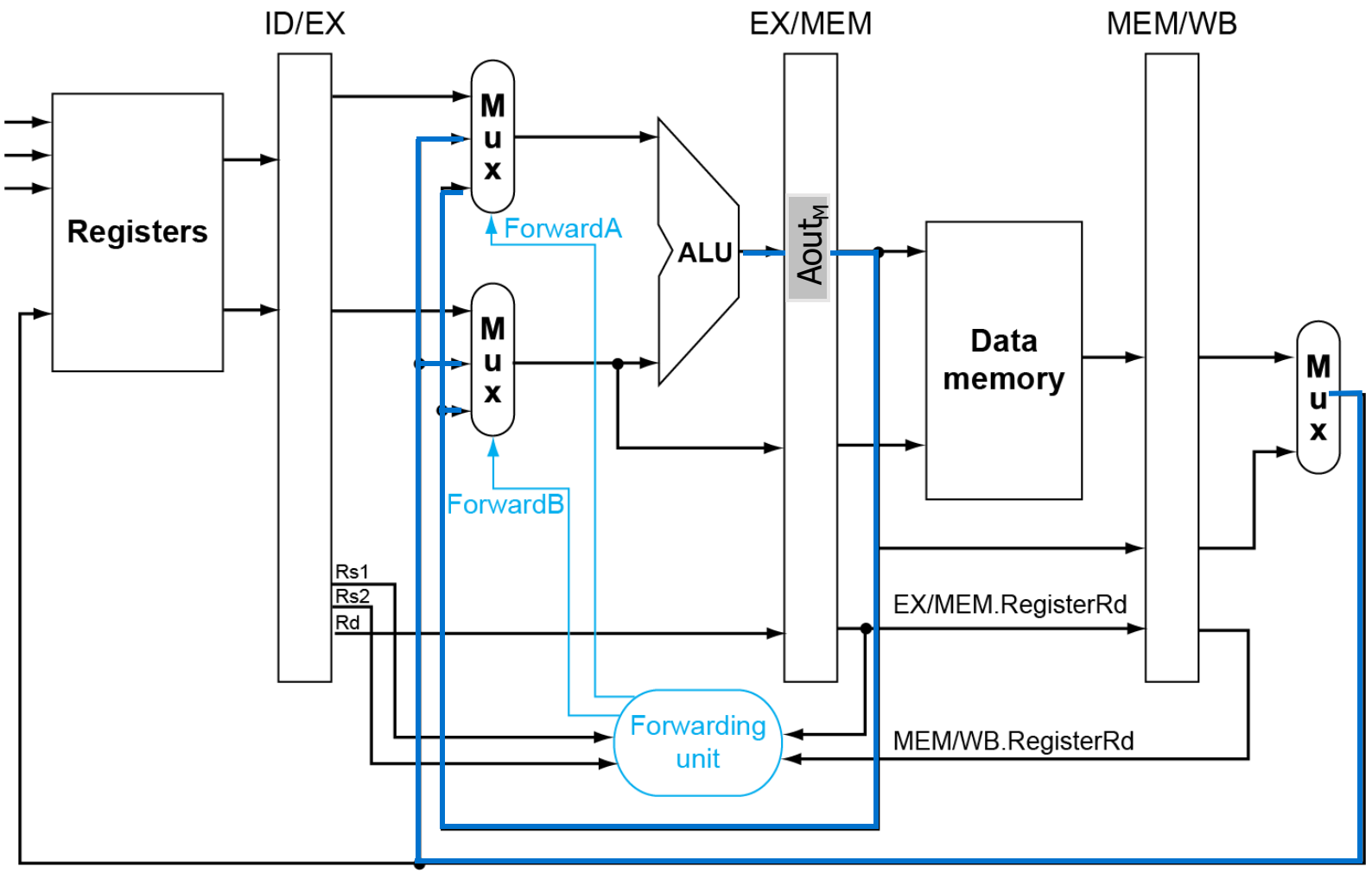
Forward from `EX/MEM` pipeline reg

Forward from `MEM/WB` pipeline reg

Detecting the Need to Forward (cont'd)

- But only if forwarding instruction will write to a register!
  - `EX/MEM.RegWrite, MEM/WB.RegWrite`
- And only if **`Rd`** for that instruction is not **`x0`**
  - `EX/MEM.RegisterRd ≠ 0, MEM/WB.RegisterRd ≠ 0`

Forwarding Paths



Forwarding Conditions

- EX hazard

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
   and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
   forwardA = 10
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
   and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
   forwardB = 10
```

- MEM hazard

```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
   and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
   forwardA = 01
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
   and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
   forwardB = 01
```

Forwarding Control

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result
ForwardB = 00	ID/EX	The second ALU operand comes from the register file
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result

Double Data Hazard

- Consider the sequence:

```
add x1,x1,x2
add x1,x1,x3
add x1,x1,x4
```

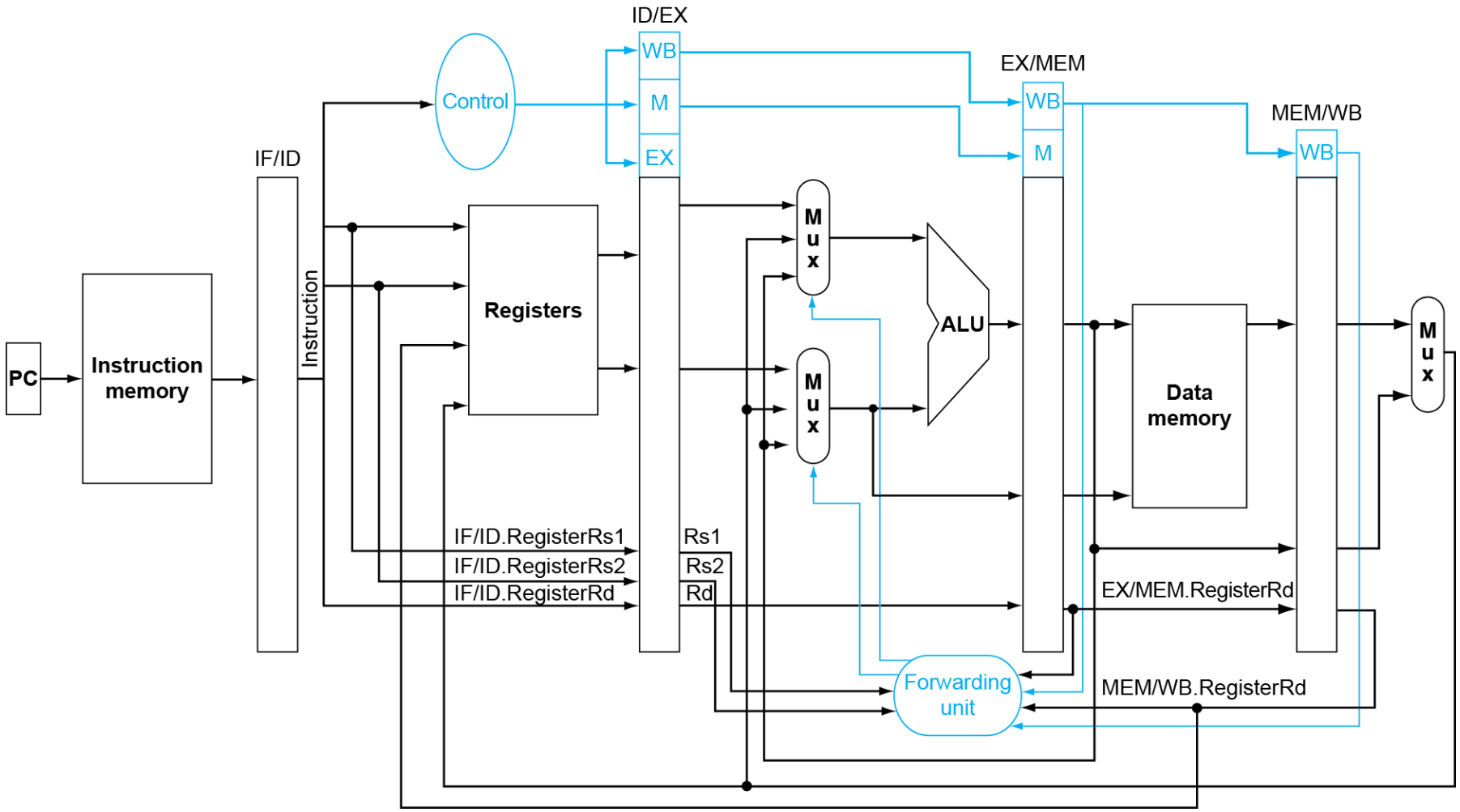
- Both hazards occur
  - Want to use the most recent
- Revise MEM hazard condition
  - Only fwd if EX hazard condition isn't true

Revised Forwarding Condition

- MEM hazard

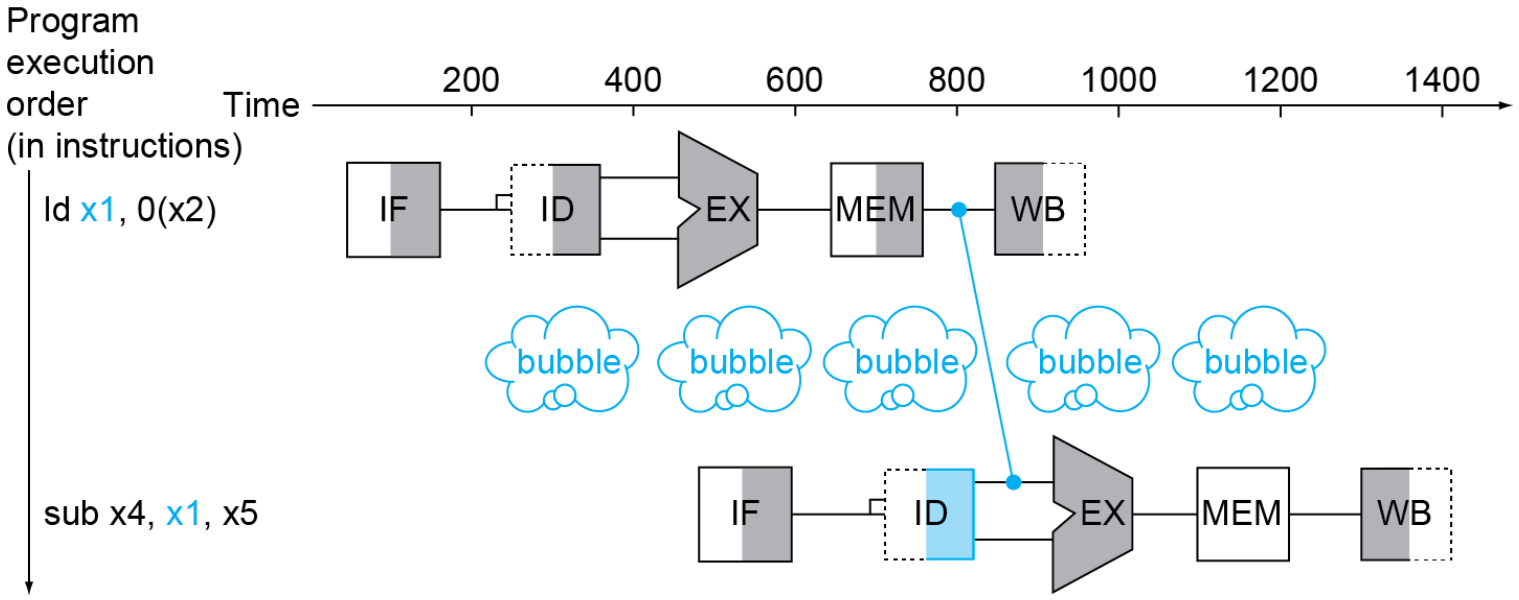
```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
   and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
       and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
   and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
   forwardA = 01

if (MEM/WB.RegWrite and (MEM/WB.RegisterRd != 0)
   and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
       and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
   and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
   forwardB = 01
```

Datapath with Forwarding



Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!



Load-Use Hazard Detection

- Check when using instruction is decoded in `ID` stage
- ALU operand register numbers in `ID` stage are given by
  - `IF/ID.RegisterRs1, IF/ID.RegisterRs2`
- Load-use hazard when

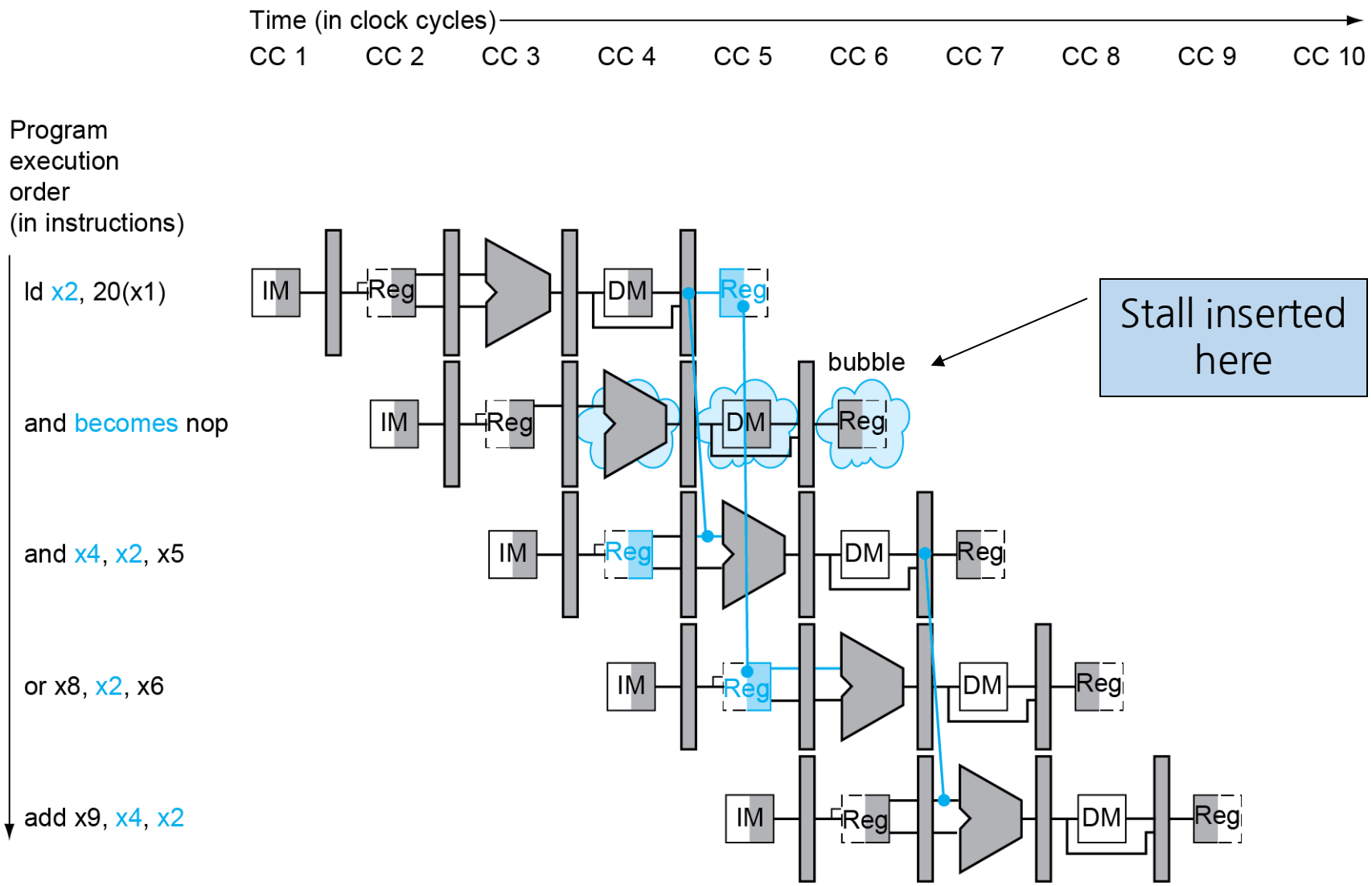
```
ID/EX.MemRead and
  ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or
   (ID/EX.RegisterRd = IF/ID.RegisterRs1))
```

- If detected, stall and insert bubble

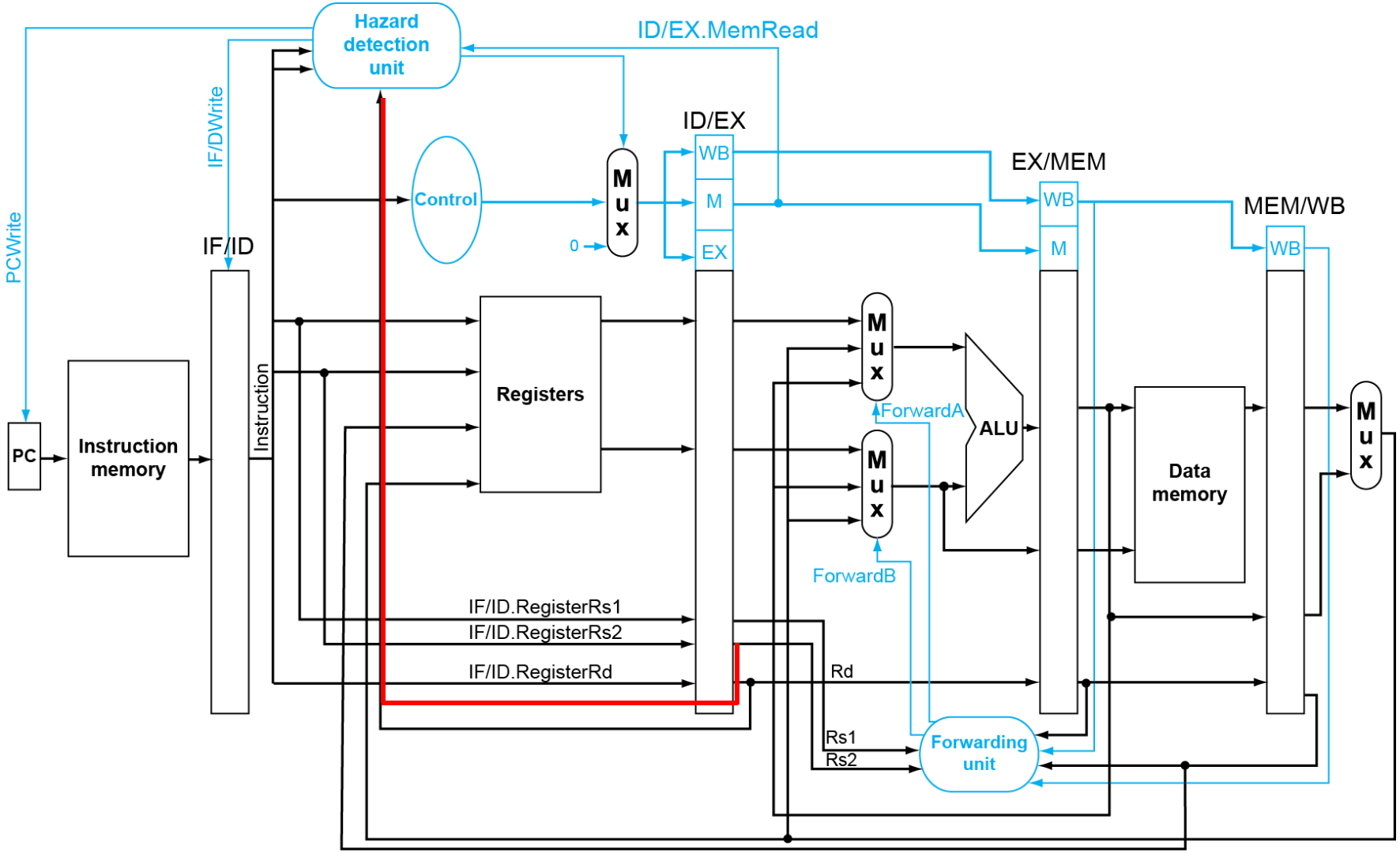
How to Stall the Pipeline

- Force control values in `ID/EX` register to `0`
  - `EX`, `MEM` and `WB` do `nop` (no-operation)
- Prevent update of `PC` and `IF/ID` register
  - Using instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows `MEM` to read data for `ld`
    - Can subsequently forward to `EX` stage

Stall/Bubble in the Pipeline



Datapath with Hazard Detection



Stalls and Performance

- Stalls reduce performance
  - But are required to get correct results
- Compiler can arrange code to avoid hazards and stalls
  - Requires knowledge of the pipeline structure

Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for `A = B + E; C = B + F;`

```
ld    x1, 0(x0)
ld    x2, 8(x0)
add   x3, x1, x2
sd    x3, 24(x0)
ld    x4, 16(x0)
add   x5, x1, x4
sd    x5, 32(x0)
```

stall

stall

13 cycles

```
ld    x1, 0(x0)
ld    x2, 8(x0)
ld    x4, 16(x0)
add   x3, x1, x2
sd    x3, 24(x0)
add   x5, x1, x4
sd    x5, 32(x0)
```

11 cycles

Other Data Hazards

```
sub   x4, x1, x3
and   x3, x5, x6
xor   x3, x7, x8
```

Write after Read (WAR)

Write after Write (WAW)

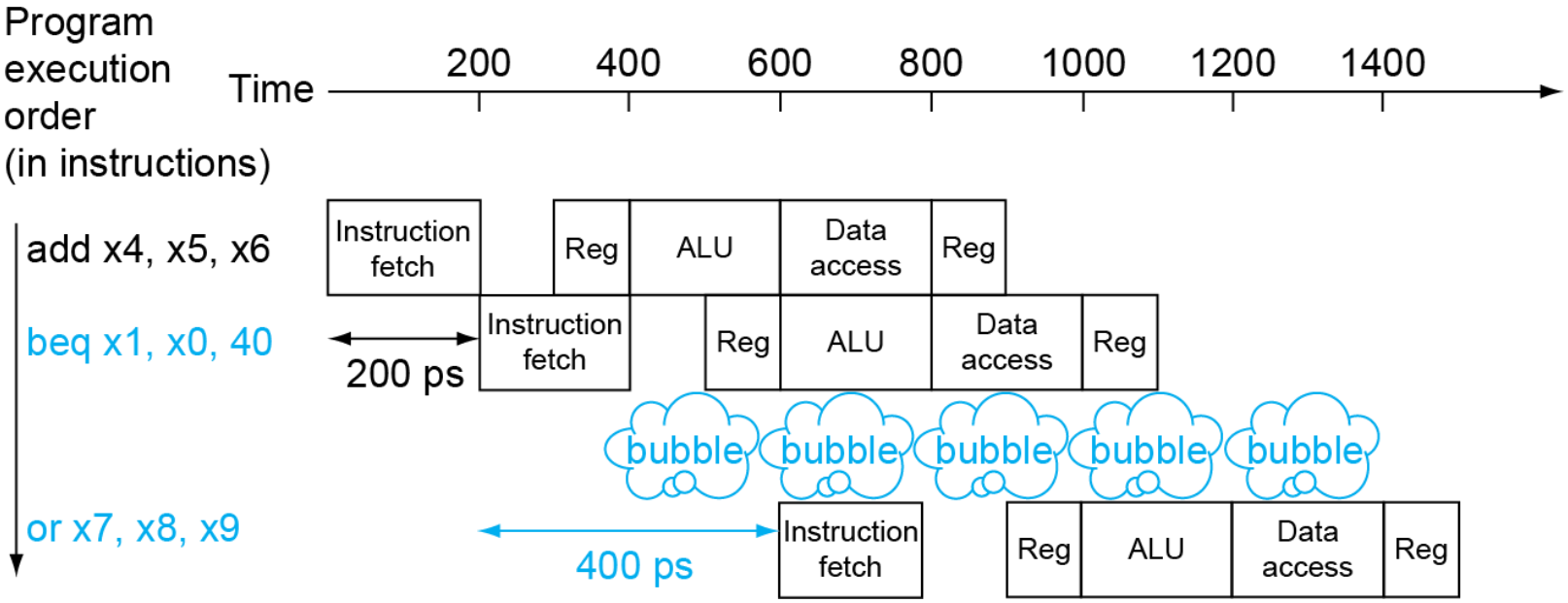
- When can WAR and WAW cause performance loss?

Control Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline can’t always fetch correct instruction
    - Still working on ID stage of branch
- In RISC-V pipeline
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage

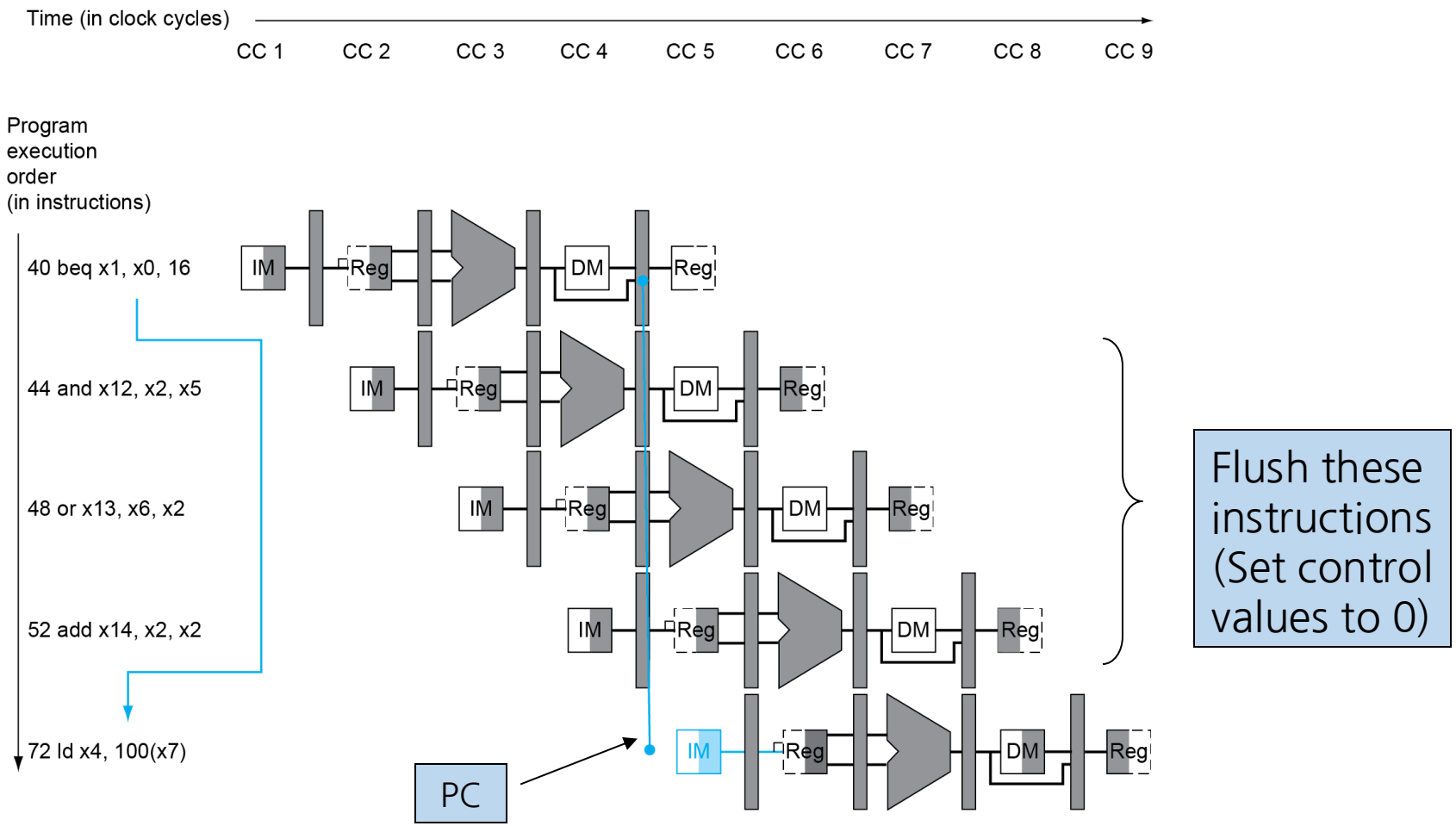
Stall on Branch

- Wait until branch outcome determined before fetching next instruction



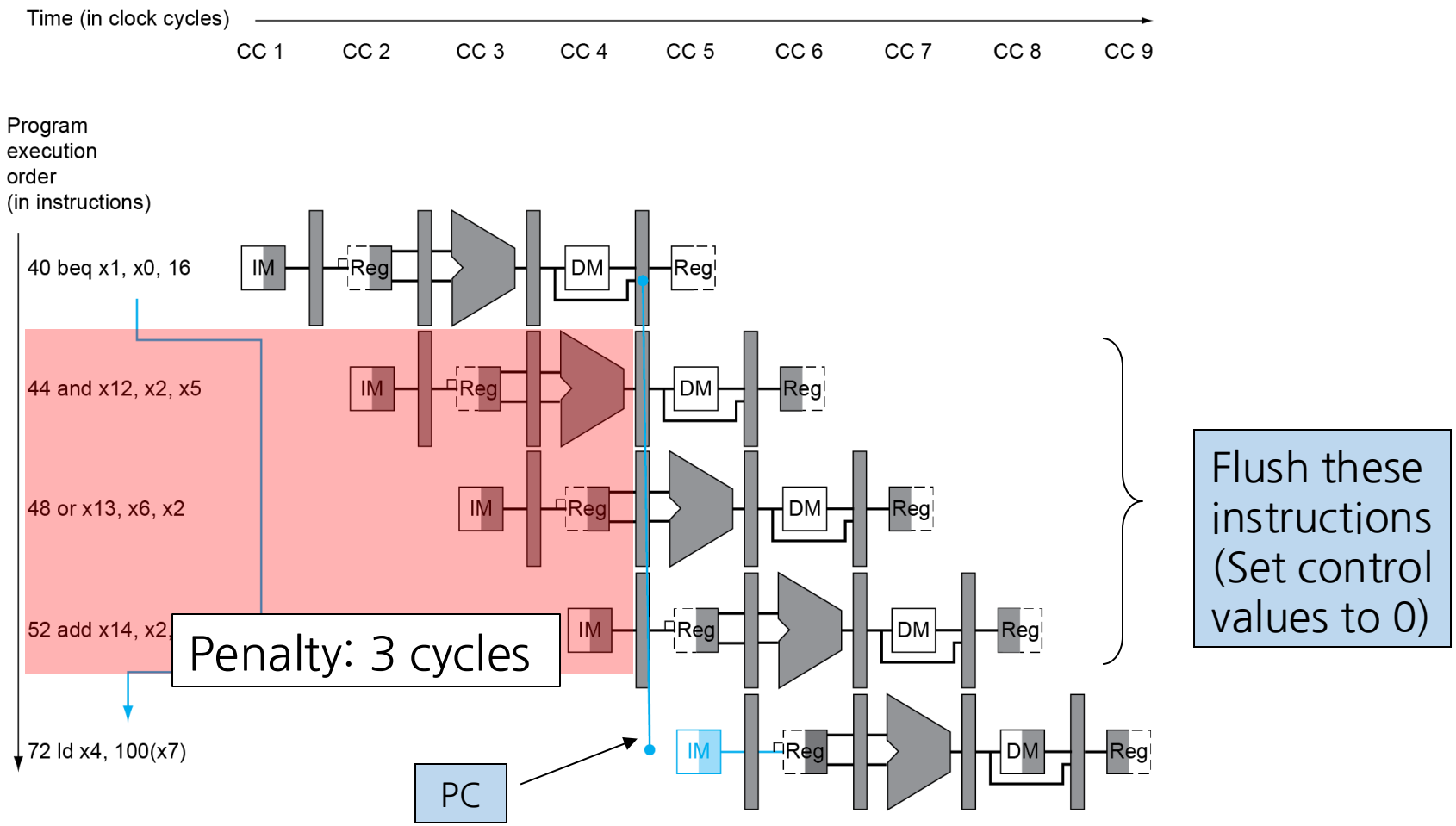
Branch Hazards

- If branch outcome determined in MEM



Branch Hazards

- If branch outcome determined in MEM

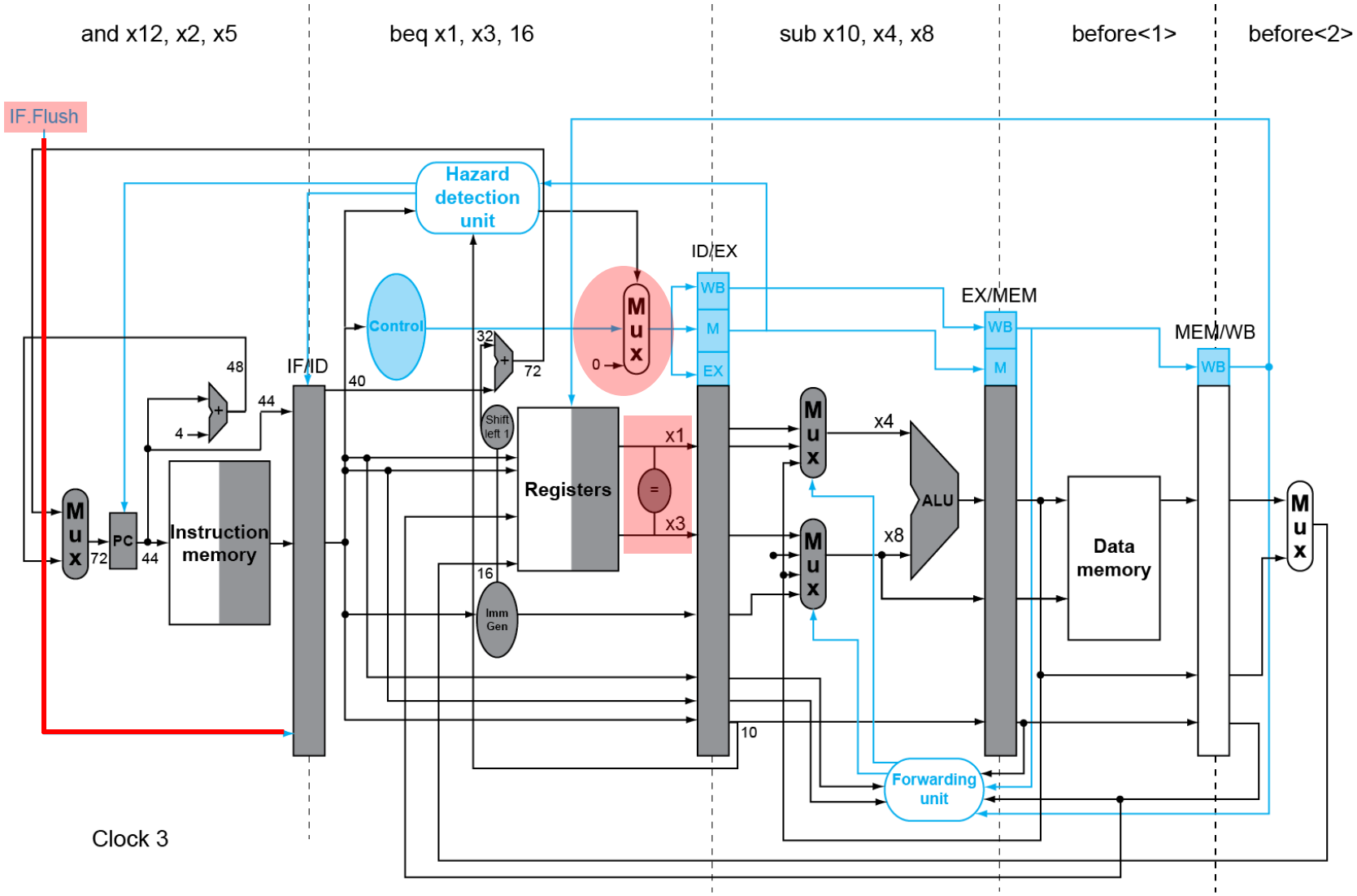


Reducing Branch Delay

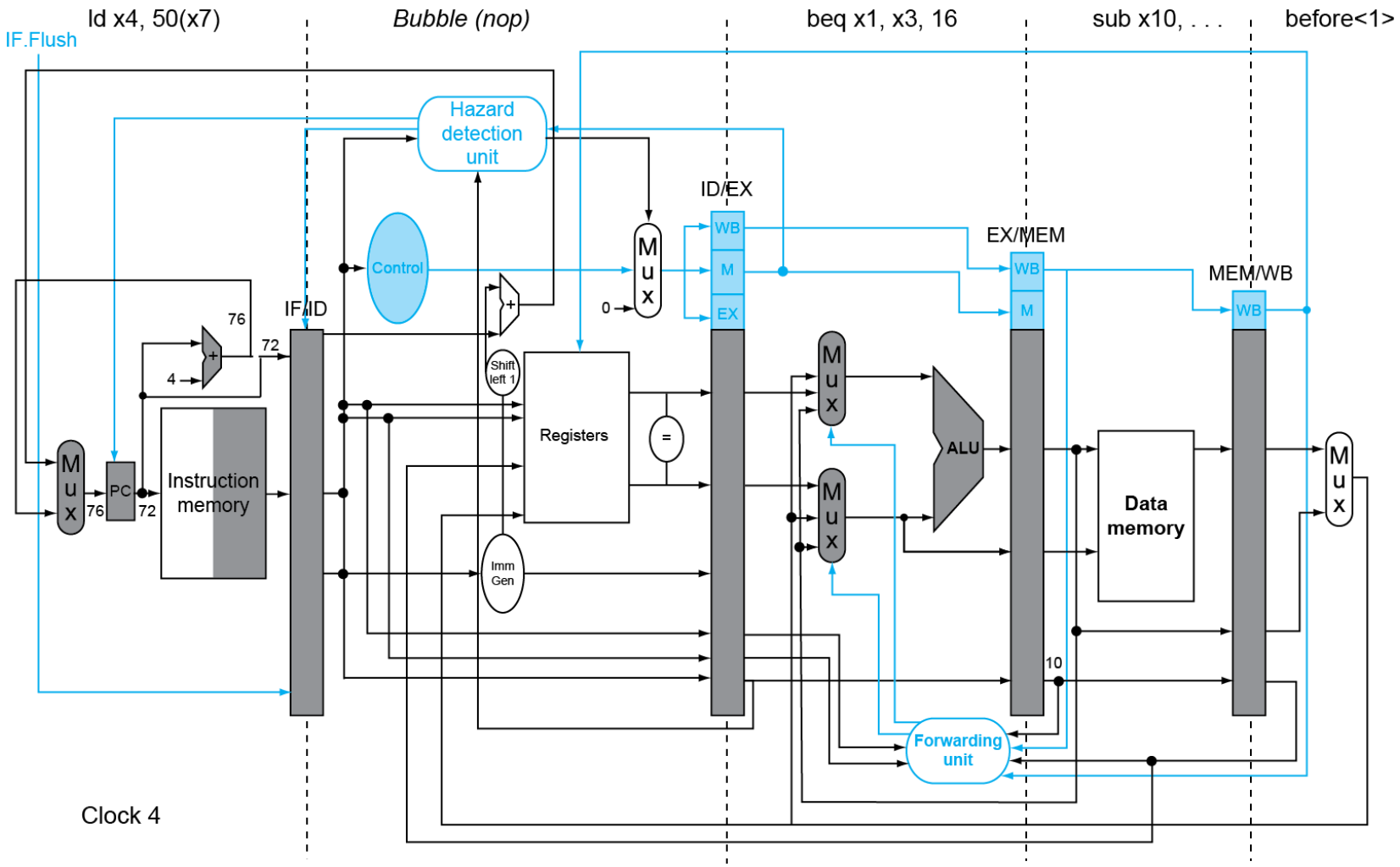
- Move hardware to determine outcome to ID stage
  - Target address adder
  - Register comparator
- Example: branch taken

```
36:  sub  x10, x4, x8
40:  beq  x1,  x3, 16   // PC-relative branch
                        // to 40+16*2=72
44:  and  x12, x2, x5
48:  orr  x13, x2, x6
52:  add  x14, x4, x2
56:  sub  x15, x6, x7
     ...
72:  ld   x4, 50(x7)
```

Example: Branch Taken

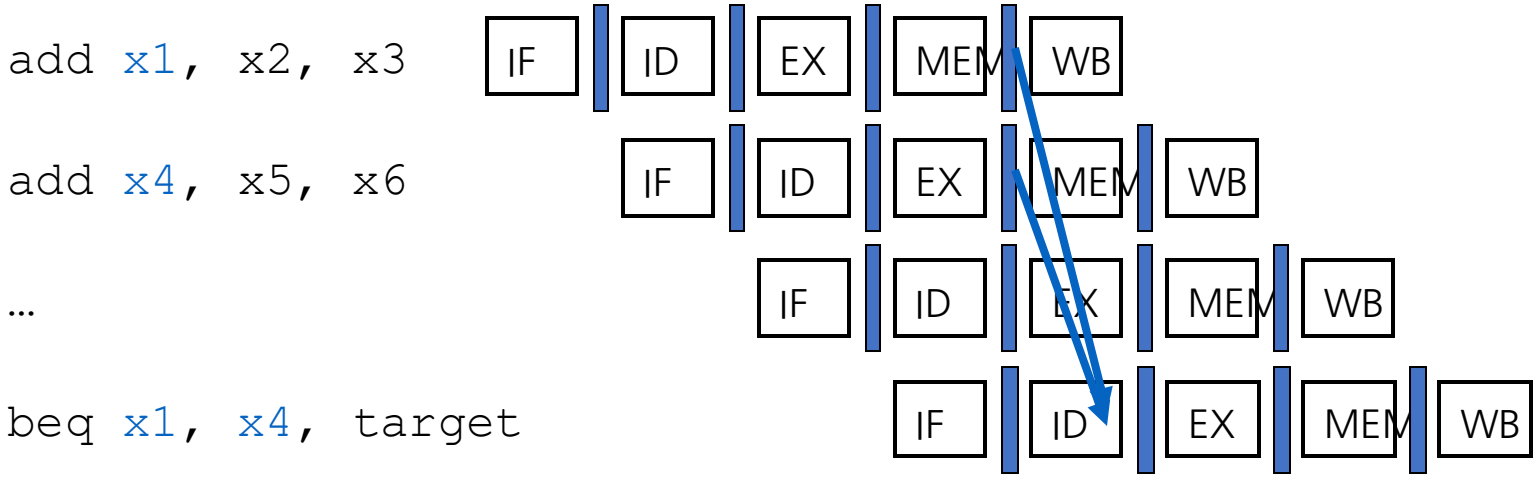


Example: Branch Taken



Data Hazards for Branches

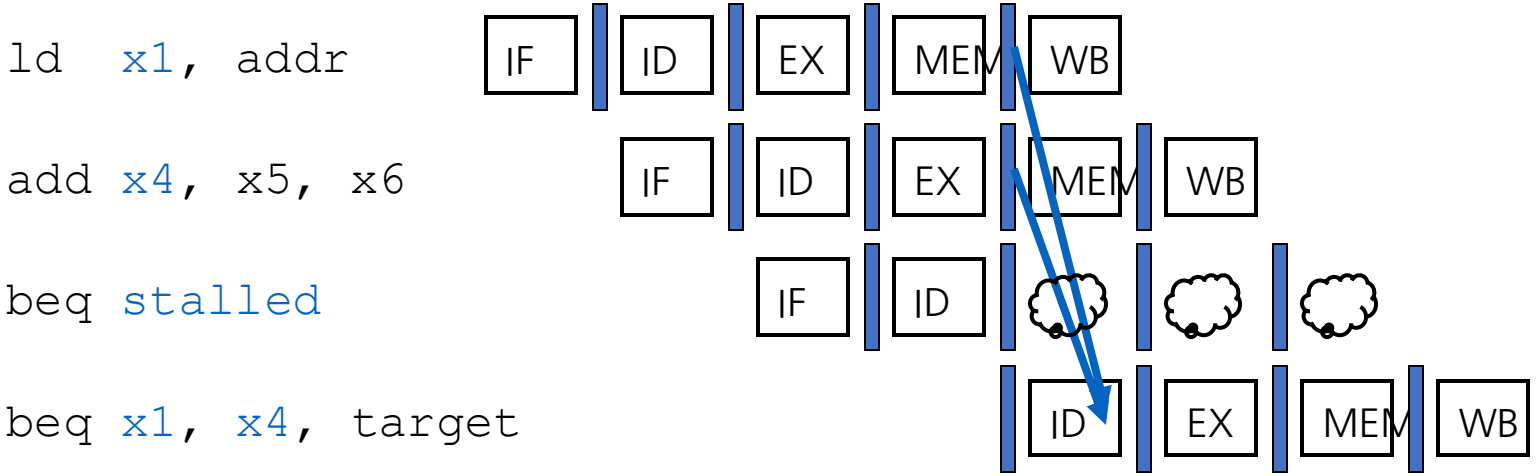
- If a comparison register is a destination of 2nd or 3rd preceding ALU instruction



- Can resolve using forwarding

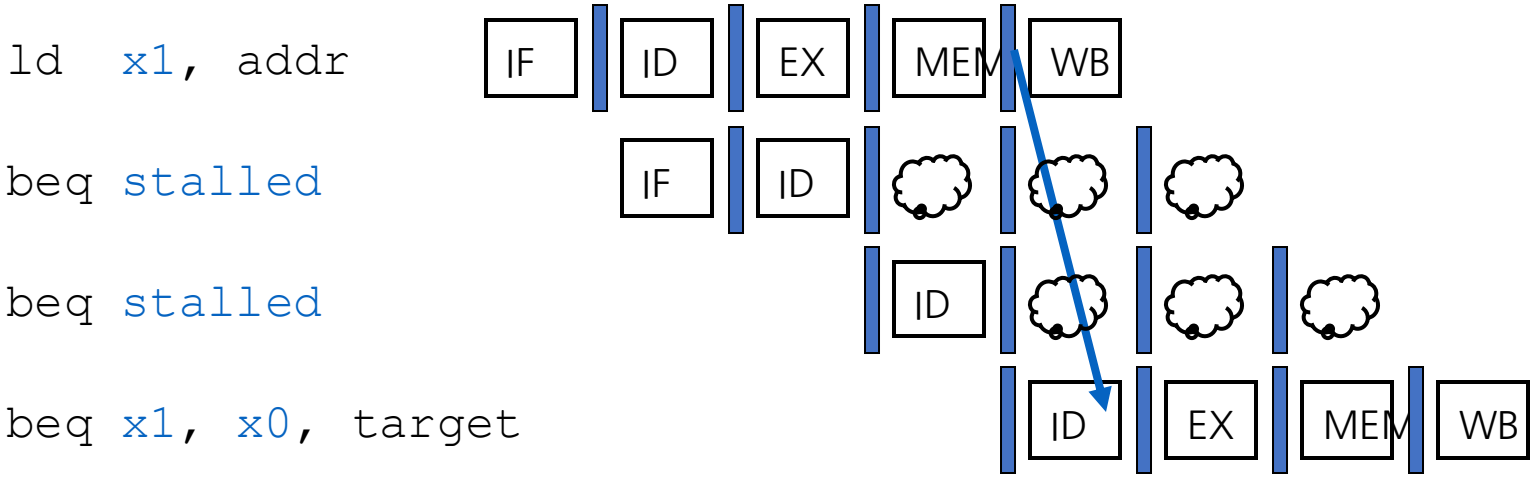
Data Hazards for Branches

- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction
  - Need 1 stall cycle



Data Hazards for Branches

- If a comparison register is a destination of immediately preceding load instruction
  - Need 2 stall cycles



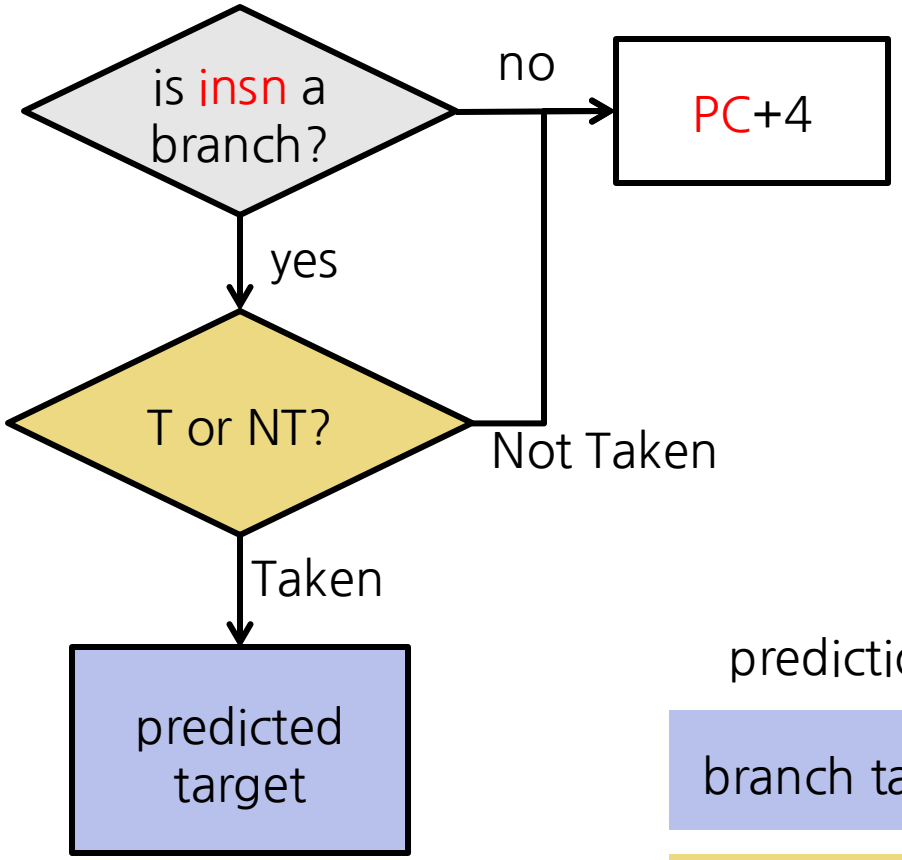
Branch Prediction

- Longer pipelines can’t readily determine branch outcome early
  - Stall penalty becomes unacceptable
- Predict outcome of branch
  - Only stall if prediction is wrong
- In RISC-V pipeline
  - Can predict branches not taken
  - Fetch instruction after branch, with no delay

Dynamic Branch Prediction

- In deeper and superscalar pipelines, branch penalty is more significant
- Use dynamic prediction
  - Branch prediction buffer (aka branch history table)
  - Indexed by recent branch instruction addresses
  - Stores outcome (taken/not taken)
- To execute a branch
  - Check table, expect the same outcome
  - Start fetching from fall-through or target
  - If wrong, flush pipeline and flip prediction

Branch Prediction Steps



- Which instructions behavior are we trying to predict?
- Where does PC come from?

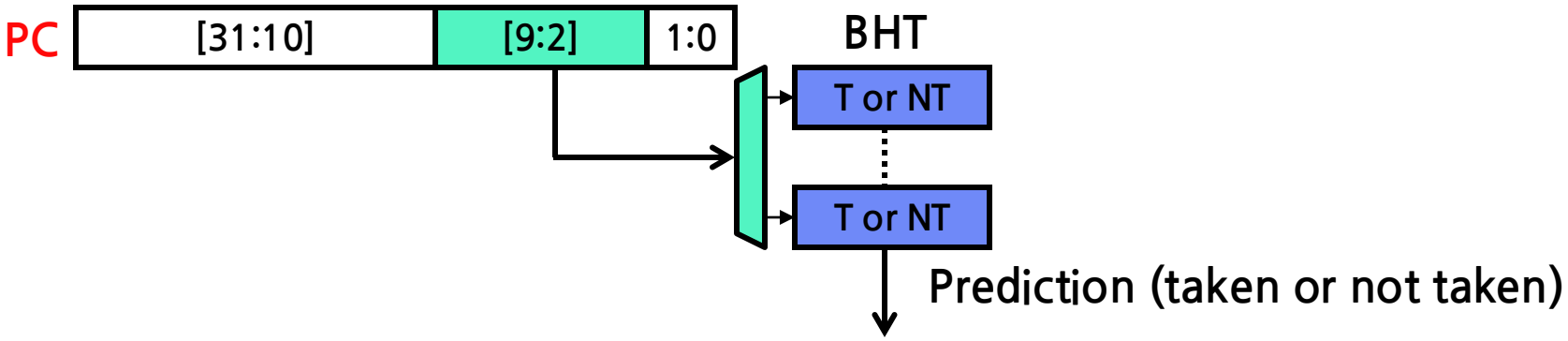
prediction source:

branch target buffer

direction predictor

Branch Direction Prediction

- **Direction predictor (DIRP)**
  - Map conditional-branch PC to taken/not-taken (T/N) decision
- **Branch history table (BHT)**: simplest predictor
  - PC indexes table of bits (0 = N, 1 = T), no tags
  - Essentially: branch will go same way it went last time



Branch History Table (BHT)

- **Branch history table (BHT)**: simplest direction predictor
  - PC indexes table of bits (0 = N, 1 = T), no tags
  - Essentially: branch will go same way it went last time
  - Problem: **inner loop branch** below

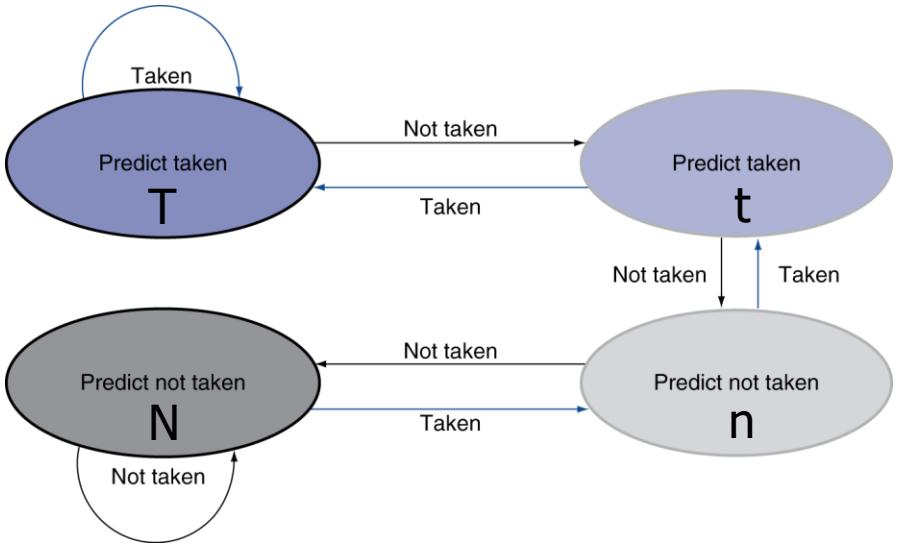
```
for (i=0;i<100;i++)
    for (j=0;j<3;j++)
        // loop body
```

    - Two "built-in" mis-predictions per inner loop iteration
    - Branch predictor "changes its mind too quickly"

Time	State	Prediction	Outcome	Result?
1	N	N	T	Wrong
2	T	T	T	Correct
3	T	T	T	Correct
4	T	T	N	Wrong
5	N	N	T	Wrong
6	T	T	T	Correct
7	T	T	T	Correct
8	T	T	N	Wrong
9	N	N	T	Wrong
10	T	T	T	Correct
11	T	T	T	Correct
12	T	T	N	Wrong

Two-Bit Saturating Counters (2bc)

- **Two-bit saturating counters**
  - Replace each single-bit prediction
    - (0,1,2,3) = (N,n,t,T)
  - Adds "hysteresis"
    - Force predictor to mis-predict twice before "changing its mind"
  - One misprediction each loop execution (rather than two)



Time	State	Prediction	Outcome	Result?
1	N	N	T	Wrong
2	n	N	T	Wrong
3	t	T	T	Correct
4	T	T	N	Wrong
5	t	T	T	Correct
6	T	T	T	Correct
7	T	T	T	Correct
8	T	T	N	Wrong
9	t	T	T	Correct
10	T	T	T	Correct
11	T	T	T	Correct
12	T	T	N	Wrong

Branch Target Address Prediction

- **Branch target buffer (BTB)**:
  - A small hardware table
  - is-a-branch = (BTB[hash(PC)].tag == PC) ? 1 : 0
  - predicted-target = (BTB[hash(PC)].tag == PC) ? BTB[PC].**target** : 0

index	tag	target
0	0	0
1	0x4e3745	0x4738da
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	0	0

Security Issue

Meltdown and Spectre

https://meltdownattack.com

Apps Google ENDIC NPR Hacker News Housekeeping Voca [MICRO14 extend... CHASE

Meltdown and Spectre

Vulnerabilities in modern computers leak passwords and sensitive data.

Meltdown and Spectre exploit critical vulnerabilities in modern processors. These
to steal data which is currently processed on the computer. While programs are
other programs, a malicious program can exploit Meltdown and Spectre to get
other running programs. This might include your passwords stored in a passw
photos, emails, instant messages and even business-critical documents.

Meltdown and Spectre work on personal computers, mobile devices, and in the
infrastructure, it might be possible to steal data from other customers.

Meltdown

Meltdown breaks the most fundamental isolation between user applications and the operating system. This attack allows a program to access the memory, and thus also the secrets, of other programs and the operating system.

Spectre breaks t
applications. It a
programs, which
secrets. In fact, t
actually increase the attack surface and may make

Spectre (security vulnerability)

https://en.wikipedia.org/wiki/Spectre_(security_vulnerability)

Apps Google ENDIC NPR Hacker News Housekeeping Voca [MICRO14 extend... CHASE » Other Bookmarks

Not logged in Talk Contributions Create account Log in

Article Talk Read Edit View history Search Wikipedia

WIKIPEDIA
The Free Encyclopedia

Main page
Contents
Featured content
Current events
Random article
Donate to Wikipedia
Wikipedia store

Interaction
Help
About Wikipedia
Community portal
Recent changes
Contact page

Tools
What links here
Related changes
Upload file
Special pages
Permanent link
Page information
Wikidata item
Cite this page

This November is the Wikipedia Asian Month. Come join us.

Spectre (security vulnerability)

From Wikipedia, the free encyclopedia

Spectre is a vulnerability that affects modern microprocessors that perform branch prediction.[1][2][3] On most processors, the speculative execution resulting from a branch misprediction may leave observable side effects that may reveal private data to attackers. For example, if the pattern of memory accesses performed by such speculative execution depends on private data, the resulting state of the data cache constitutes a side channel through which an attacker may be able to extract information about the private data using a timing attack.[4][5][6]

Two Common Vulnerabilities and Exposures IDs related to Spectre, CVE-2017-5753 (bounds check bypass, Spectre-V1, Spectre 1.0) and CVE-2017-5715 (branch target injection, Spectre-V2), have been issued.[7] JIT engines used for JavaScript were found to be vulnerable. A website can read data stored in the browser for another website, or the browser's memory itself.[8]

On March 15, 2018, Intel reported that it will redesign its CPUs (performance losses to be determined) to help protect against the Spectre and related Meltdown vulnerabilities (especially, Spectre variant 2 and Meltdown, but not Spectre variant 1), and expects to release the newly redesigned processors later in 2018.[9][10][11][12] On October 8, 2018, Intel is reported to have added hardware and firmware mitigations regarding Spectre and Meltdown vulnerabilities to its latest processors.[13] On October 18, 2018, MIT researchers suggested a new mitigation approach, called DAWG (Dynamically Allocated Way Guard), which may promise better security without compromising performance.[14]

SPECTRE

A logo created for the vulnerability, featuring a ghost with a branch

https://meta.wikimedia.org/wiki/Wikipedia_Asian_Month_2018

Pipeline Summary

- Pipelining improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards
  - Structure, data, control
- Instruction set design affects complexity of pipeline implementation